Prototipo de sistema de traducción de Lengua de Señas Argentina con procesamiento en tiempo pseudo-real

Maite Nigro Francisco Veron

Director: Esp. Ing. Gonzalo Avalos Ribas

Co-director: Ing. Bruno Constanzo

Universidad Nacional de Mar del Plata
Facultad de Ingeniería
Departamento de Ingeniería en Informática

Octubre 2026

Agradecimientos

De Maite: Para la Flaqui.
De Francisco: Para la facultad de Ingeniería de Mar del Plata, y sus docentes que, con todo su cariño y pasión, nos enseñaron a lo largo de estos años. Para la familia, amigos, personas que pasaron por mi camino, sin todos ellos no estaría donde estoy hoy. Para Carmen Beatríz Loyola
A Miriam Rolls, por su apoyo y ayuda en este proyecto.

Índice general

1. Introducción **1**
- 1.1. Contexto . . . 1
- 1.2. Motivación . . . 1
- 1.3. Problemática . . . 1
- 1.4. Alcance y limitaciones . . . 2
  - 1.4.1. Delimitación del vocabulario y asesoramiento lingüístico . . . 2
  - 1.4.2. Límites operativos y comportamiento ante entradas no contempladas 3
  - 1.4.3. Infraestructura de despliegue . . . 4

2. Objetivos **5**
- 2.1. Objetivo general . . . 5
- 2.2. Objetivos específicos . . . 5

3. Marco teórico, estado del arte y metodología **7**
- 3.1. Marco teórico . . . 7
  - 3.1.1. Componentes articulatorios y estructura fonológica de la LSA . . . 7
  - 3.1.2. Estructura lingüística de la LSA y orden manual del prototipo . . . 7
  - 3.1.3. Sistemas de tiempo real y tiempo pseudo-real . . . 8
  - 3.1.4. Visión computacional y reconocimiento basado en esqueletos . . . 8
  - 3.1.5. Modelado de secuencias temporales y atención . . . 9
  - 3.1.6. Modelos de lenguaje pequeños y adaptación *LoRA* . . . 10
  - 3.1.7. Estándares de captura web y arquitecturas de extensión web . . . 11
- 3.2. Estado del arte . . . 11
- 3.3. Metodología de desarrollo . . . 12
  - 3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial . . 14
- 3.4. Herramientas y tecnologías empleadas . . . 15

4. Análisis de requerimientos **16**
- 4.1. Requerimientos funcionales . . . 16
- 4.2. Requerimientos no funcionales . . . 17
- 4.3. Restricciones . . . 17
  - 4.3.1. Especificación del entorno de ejecución cliente . . . 18
- 4.4. Casos de uso . . . 19
  - 4.4.1. Casos de uso esenciales en formato breve . . . 19
  - 4.4.2. Diagrama de casos de uso . . . 20
  - 4.4.3. Casos de uso expandidos . . . 20

5. Diseño del Sistema **26**
5.1. Arquitectura del sistema . . . 26
5.1.1. Justificación de la partición y modelo de privacidad . . . 26
5.1.2. Flujo integral de datos entre componentes . . . 26
5.2. Diseño de contratos de API REST . . . 27
5.2.1. API local del motor ILSA . . . 27
5.2.2. API del servicio semántico remoto . . . 28
5.2.3. API del servicio de archivos del avatar . . . 28
5.3. Diseño de la representación gestual inversa . . . 28
5.3.1. Procesamiento geométrico de coordenadas y segmentación temporal 30
5.3.2. Preparación, acondicionamiento y ensamblado cinemático . . . 30
5.3.3. Estrategia de contingencia léxica . . . 32
5.4. Diseño de la extensión de videollamada . . . 32
5.5. Patrones de diseño de software aplicados . . . 32
5.6. Diseño conceptual de la arquitectura escalable de producción en la nube . . 33
5.6.1. Ingreso . . . 34
5.6.2. Traductor . . . 34
5.6.3. Archivos de seña . . . 34
5.6.4. Monitoreo . . . 34
5.6.5. Qué queda fuera de las pruebas . . . 34

6. Implementación y Desarrollo **35**
6.1. Estructura del código . . . 35
6.1.1. Organización del Repositorio . . . 35
6.1.2. Descripción de Componentes y Responsabilidades . . . 36
6.1.3. Convenciones de Nomenclatura y Estándares . . . 36
6.1.4. Archivos de Configuración y Entorno de Pruebas . . . 37
6.2. Implementación del subsistema de reconocimiento gestual . . . 37
6.2.1. Preprocesamiento de datos y pipeline de extracción . . . 37
6.2.2. Arquitectura neuronal TinySkeletonClassifier . . . 37
6.3. Implementación del subsistema de traducción semántico bidireccional . . . 38
6.3.1. Selección y especificación de la suite de modelos de lenguaje . . . 38
6.3.2. Régimen de entrenamiento y ajuste LoRA . . . 39
6.3.3. Canal directo (LSA → Español) . . . 40
6.3.4. Canal inverso (Español → LSA) . . . 41
6.3.5. Cuantización GGUF Q4_K_M y estimación de memoria . . . 42
6.4. Integración en Google Meet y arquitectura de la extensión . . . 42
6.4.1. Secuencia completa de inferencia y comunicación multidireccional . 42
6.4.2. Motor local ILSA (FastAPI) . . . 45
6.5. Implementación del avatar . . . 45
6.5.1. Vía descartada: retarget hacia un avatar VRM . . . 45
6.5.2. Decisiones técnicas de la reproducción . . . 46
6.6. Instancia de validación del prototipo mediante microservicios en la nube . . 47

7. Pruebas y validación **49**
7.1. Plan de pruebas . . . 49
7.1.1. Alcance y tipos de pruebas . . . 49
7.1.2. Entornos de evaluación . . . 50
7.1.3. Criterios de aceptación y trazabilidad . . . 50

7.2. Resultados de la verificación . . . 50
7.2.1. Evaluación del clasificador gestual (TinySkeletonClassifier) . . . 50
7.2.2. Resultados de los módulos de traducción semántica . . . 53
7.2.3. Pruebas de la lógica de control del motor local . . . 54
7.2.4. Tratamiento de fallas de integración en Google Meet . . . 54
7.3. Métricas de calidad . . . 55
7.3.1. Pruebas de latencia y end-to-end en llamada real . . . 55
7.3.2. Evaluación del uso de recursos de hardware en el dispositivo cliente 56
7.3.3. Consumo de memoria y huella del sistema en servidor . . . 57
7.3.4. Comparativa de rendimiento según arquitectura de hardware . . . 57

8. Evaluación **59**
8.1. Cumplimiento de objetivos . . . 59
8.1.1. Evaluación del objetivo general . . . 59
8.1.2. Evaluación de los objetivos específicos . . . 59
8.2. Limitaciones . . . 61
8.3. Riesgos detectados y estrategias de mitigación . . . 62

9. Conclusiones y trabajos futuros **64**
9.1. Conclusiones . . . 64
9.1.1. Síntesis del cumplimiento de objetivos . . . 64
9.1.2. Aportes técnicos e ingenieriles . . . 65
9.1.3. Impacto social y marco regulatorio . . . 65
9.2. Trabajos futuros . . . 65
9.2.1. Inclusión de la comunidad sorda y validación participativa . . . 66
9.2.2. Ampliación del dataset y captura multicámara . . . 66
9.2.3. Incorporación de rasgos no manuales y reconocimiento continuo . . 66
9.2.4. Evolución hacia un avatar tridimensional (3D) . . . 66
9.2.5. Despliegue del clúster elástico multinodo en la nube . . . 66
9.2.6. Generalización a nuevas plataformas y ejecución local de SLM . . . 66
9.2.7. Aportes de la inteligencia artificial generativa en el ciclo de desarrollo 67

A. Vocabulario del clasificador **71**

B. Ramas de entrenamiento y de servicios **72**
B.1. Repositorio de entrenamiento del clasificador . . . 72
B.2. Servicio semántico en la nube . . . 72
B.3. Entrenamiento del canal directo . . . 73
B.4. Entrenamiento del canal inverso . . . 73
B.5. Servicio de poses del avatar . . . 73

C. Arquitectura escalable de producción en la nube **74**

Resumen

En la actualidad, la interacción virtual entre personas sordas y oyentes se limita casi exclusivamente a la comunicación escrita o a la transcripción unilateral de voz a texto, lo que genera una marcada asimetría en la comunicación expresiva durante videollamadas. Este Trabajo Final de Grado desarrolla un prototipo de traducción bidireccional entre la Lengua de Señas Argentina (LSA) y el español oral estructurado bajo una arquitectura híbrida de computación en el borde y servicios en la nube con procesamiento en tiempo pseudo-real.

El sistema articula dos canales comunicacionales complementarios. En el sentido señante-oyente, la computadora del usuario captura localmente el flujo de video desde la cámara web, extrae coordenadas articulares tridimensionales mediante visión computacional y clasifica un vocabulario de 100 señas mediante una red neuronal ligera (Conv1D + Tiny Transformer con *attention pooling*). Con el fin de evitar la saturación de memoria y cómputo que provocaría ejecutar modelos de lenguaje generativos en dispositivos convencionales, las glosas clasificadas son enviadas hacia un backend semántico en la nube que transforma la secuencia en oraciones fluidas en español rioplatense, las cuales retornan al cliente para su proyección en subtítulos o síntesis de voz. En sentido inverso (oyente-señante), el sistema procesa la voz y/o texto del interlocutor oyente mediante un módulo semántico inverso en la nube que estructura las glosas, enviándolas a un avatar en el cliente que ejecuta las animaciones correspondientes o recurre al deletreo dactilológico para términos fuera del léxico. El módulo clasificador gestual local alcanza un 99,0 % de exactitud en validación fuera de línea y un 99,6 % de acierto en cámara real, con iluminación adecuada. Por su parte, el modelo elegido para el servicio, Qwen2.5 1,5B, adaptado mediante LoRA y cuantizado a GGUF `Q4_K_M`, alcanza un acierto exacto del 94,96 % en el canal directo y del 91,18 % en el inverso. La generación local en CPU de ese modelo es de 256,30 ms y de 290,82 ms. En la sesión integrada, el tiempo medio de inferencia remota es de 1444,4 ms en el canal directo y de 1368,2 ms en el inverso. Como mecanismo de integración para videollamadas, el sistema se acopla a Google Meet mediante una extensión de navegador conectada al motor local ILSA, y asegura la privacidad del usuario al no transmitir video crudo fuera del dispositivo cliente. Finalmente, se deja documentada la propuesta de diseño de una arquitectura en la nube escalable para soportar la demanda concurrente en entornos de producción.

Palabras clave:

Lengua de Señas Argentina, reconocimiento de gestos, arquitectura híbrida cliente-nube, traducción semántica bidireccional, avatar esquelético, accesibilidad.

Capítulo 1

Introducción

1.1. Contexto

Tradicionalmente, la sociedad abordó a la comunidad sorda desde una perspectiva clínica centrada en la rehabilitación y la corrección auditiva. Ese enfoque omite que la comunidad sorda se identifica, ante todo, como una minoría lingüística con cultura y gramática propias, independientes del español oral. En la Argentina, la Lengua de Señas Argentina (LSA) cuenta con reconocimiento legal formal a través de la Ley 27.710 [1], la cual garantiza su preservación y el derecho de las personas sordas a comunicarse en su lengua natural en todos los ámbitos sociales.

En el contexto digital contemporáneo, las plataformas de videoconferencia se han convertido en herramientas centrales para el ámbito laboral, educativo y de la vida cotidiana. Sin embargo, los *software* de uso masivo presentan escasa integración de herramientas automatizadas que admitan la LSA como un flujo comunicacional bidireccional en tiempo pseudo-real. Los recursos tecnológicos existentes suelen restringirse a diccionarios estáticos o repositorios de video, carentes de capacidad de interpretación contextual e interactiva durante una llamada.

1.2. Motivación

Este proyecto nace de la aspiración de reducir las barreras comunicacionales de las personas señantes en interacciones cotidianas y de asistencia mediante herramientas informáticas. Resulta indispensable avanzar hacia esquemas de integración lingüística en los que la tecnología actúe como un puente transparente en ambos sentidos de la conversación: expresivo (señante $\rightarrow$ oyente) y receptivo (oyente $\rightarrow$ señante).

1.3. Problemática

La problemática central radica en la asimetría comunicacional presente en las plataformas de videoconferencia. Si bien las herramientas de transcripción automática de voz a texto han alcanzado un alto grado de adopción, resultan insuficientes como solución de accesibilidad integral: para diversas personas sordas, el texto escrito opera como una segunda lengua con reglas gramaticales ajenas, e incluso genera barreras de acceso para quienes no se encuentran alfabetizados en la lengua oral.

En el extremo expresivo, la comunicación mediante LSA carece de soporte nativo en el software de videollamadas comercial. La falta de módulos capaces de capturar e interpretar la LSA hacia voz o texto audible fuerza la dependencia de interpretación humana o restringe la interacción al chat escrito.

A esta brecha se añade la especificidad lingüística local: los modelos de visión y lenguaje desarrollados internacionalmente se encuentran enfocados mayoritariamente en la Lengua de Señas Americana (ASL). Las técnicas y arquitecturas neuronales resultan reutilizables, pero los modelos preentrenados y conjuntos de datos de ASL no se extrapolan directamente a la LSA debido a diferencias léxicas, dactilológicas y sintácticas, por lo que es necesaria la construcción de un corpus propio y de modelos adaptados al contexto argentino.

1.4. Alcance y limitaciones

El sistema desarrollado comprende un prototipo funcional de traducción bidireccional en tiempo pseudo-real concebido como una extensión para navegadores basados en la tecnología Chromium (como Google Chrome, Microsoft Edge, Brave, Opera o Vivaldi), articulada con un motor de ejecución en segundo plano denominado ILSA (*Intérprete de LSA*).

Para abordar la interpretación de la LSA se introduce el concepto de **glosa**: representación escrita en letras mayúsculas asignada a una seña para identificarla, y analizar su gramática sin constituir una traducción literal término a término.

El sistema opera a través de dos canales comunicacionales integrados:

- **Canal directo (Señante → Oyente):** En el cliente, el sistema toma señas aisladas de la cámara, extrae coordenadas articulares y clasifica cada glosa. Una seña empieza cuando las manos entran en el cuadro y termina cuando salen. La frase se cierra tras 4 segundos sin manos en el cuadro. Una sola glosa léxica, un deletreo de letras o una seguidilla de dígitos se resuelven en ILSA y no consultan la nube. El resto viaja como texto de glosas al servicio semántico, que devuelve una oración en español rioplatense. Con la salida Subtítulo, la oración queda en el video. Con Audio o Ambos, la voz suena para quien seña y también para el resto de los participantes de la llamada. Si el servicio no responde, ILSA muestra las glosas aceptadas en texto, sin rearmar la oración.

- **Canal inverso (Oyente → Señante):** La voz del oyente se transcribe en su computadora, o el sistema toma el texto del chat. La oración va a la nube, que la reordena según el orden manual de este prototipo. El cliente dibuja el clip de pose de cada glosa sobre un esqueleto cuando el archivo existe. Si la glosa no tiene archivo, ILSA pide cada letra y la dibuja con el alfabeto manual. Ese deletreo no asegura que el interlocutor comprenda el significado de la palabra.

1.4.1. Delimitación del vocabulario y asesoramiento lingüístico

El alcance funcional del clasificador gestual está delimitado a un vocabulario objetivo de 100 señas de LSA (cuyo detalle y categorización se presentan en el Apéndice A). Para la selección y estructuración de este corpus se contó con la colaboración y asesoramiento técnico de Miriam Rolls, profesora e intérprete de LSA, quien participó en la delimitación del recorte léxico. El vocabulario se orientó hacia situaciones de interacción cotidiana y contextos de primera respuesta o emergencia (verbos de auxilio, identificación, parentesco

y datos temporales). Esas señas se distinguen sin la gesticulación facial. La validación garantizó la fidelidad lingüística de las señas seleccionadas para la construcción del corpus, sin que esto constituya una evaluación formal de usabilidad ni un ensayo clínico con usuarios finales en entornos reales de emergencia.

1.4.2. Límites operativos y comportamiento ante entradas no contempladas

El prototipo presenta las siguientes delimitaciones operativas:

- **Delimitación de señas y de frases:** Una seña empieza cuando las manos entran en el cuadro y termina cuando salen. La frase termina cuando pasan 4 segundos sin manos en el cuadro.
- **Mano hábil:** ILSA muestra dos botones, diestro y zurdo. Diestro aparece seleccionado. La persona confirma esa opción o pasa a zurdo: siempre hay una mano elegida. Con zurdo, el vector se espeja en horizontal y se cruzan las dos manos, porque el modelo se entrenó con mano derecha.
- **Rasgos no manuales y marcas manuales:** El clasificador excluye la malla facial y no trata la boca, las cejas ni la cabeza como rasgo gramatical. Una pregunta, una negación o un tiempo solo entran al enunciado si se producen con una seña o una marca manual. El prototipo no recupera un significado que exista únicamente en la información que descarta.
- **Predicción, glosa incorporada e indicador del HUD:** Una predicción es la clase que devuelve el clasificador, con su confianza. Una glosa incorporada es la que quedó en la frase. Ver el nombre en el HUD no significa que vaya a transmitirse. El color informa el resultado:
  - **Cian, «En el mensaje: HOLA»:** la confianza es de al menos 0,75 y la glosa quedó en la frase.
  - **Coral, «Poca confianza: HOLA»:** el clasificador reconoció la seña, pero la confianza quedó bajo 0,75 y no se agregó.
  - **Gris, «No se agregó: HOLA»:** la confianza alcanzó 0,75 y un filtro de repetición la dejó fuera. Ese filtro rechaza la repetición inmediata de una glosa léxica, admite la repetición en los dígitos y admite hasta dos letras iguales seguidas.

  Solo el cian entra al enunciado. Una seña ausente del vocabulario puede parecerse a una clase conocida y salir con confianza de al menos 0,75. En ese caso el HUD la muestra en cian y la glosa incorrecta entra a la frase.
- **Reproducción del avatar:** El dibujo está garantizado para el clip cuyo archivo existe, hasta 24 archivos por enunciado. Si una glosa no tiene archivo, se deletrea letra por letra, y ese deletreo no asegura la comprensión del significado. Lo que supera los 24 archivos queda en texto y no se anima. Si el servidor de movimientos no responde, las glosas quedan en texto y el clip queda vacío. El avatar mantiene, además, una cola de hasta 20 animaciones, una por oración. Una oración nueva espera su turno y no interrumpe la animación en curso.

- **Resiliencia ante fallas de red:** Si el traductor no responde, ILSA conserva las glosas ya aceptadas y las muestra en texto, sin rearmar la oración.
- **Arranque:** Encender, Reiniciar o Apagar vuelven a la pantalla de carga. Ahí ILSA exige que respondan el traductor semántico y el servidor de poses. Si alguno falla, queda el botón Reintentar y el panel principal no se abre.

1.4.3. Infraestructura de despliegue

En la red, el alcance incluye el diseño de una arquitectura elástica, con contenedores y balanceo de carga, para varias videollamadas a la vez. Las pruebas no despliegan ese esquema. La evaluación usa una instancia de Cloud Run por servicio, en CPU.

Capítulo 2

Objetivos

2.1. Objetivo general

Desarrollar un prototipo de sistema de traducción bidireccional entre la LSA y el español rioplatense con procesamiento en tiempo pseudo-real, diseñado para operar sobre plataformas de videoconferencia mediante una arquitectura híbrida cliente-nube, con el fin de mitigar la asimetría en la comunicación expresiva y receptiva, lo que ayudaría a fomentar la autonomía de las personas sordas en la interacción con interlocutores oyentes no señantes.

2.2. Objetivos específicos

- Implementar un módulo local de visión computacional y clasificación gestual basado en la extracción de puntos de referencia corporales para el reconocimiento de un vocabulario de 100 señas de LSA.
- Diseñar e integrar el servicio semántico del canal directo (LSA → Español) para transformar secuencias de glosas aisladas en oraciones fluidas y gramaticalmente correctas en español rioplatense.
- Diseñar e integrar el servicio semántico del canal inverso (Español → LSA) para reestructurar oraciones del español en la secuencia manual de glosas que fija este prototipo, dentro del orden de la Sección 3.1.2.
- Implementar un módulo de visualización gestual inversa mediante un avatar capaz de animar el clip de cada glosa con archivo y de deletrear los términos ausentes, con el límite de garantía de la Sección 1.4.2.
- Diseñar una arquitectura escalable en la nube para el aprovisionamiento concurrente de los servicios, lo que desacopla el procesamiento del hardware cliente.
- Resguardar la privacidad de los datos biométricos de los usuarios al ejecutar la extracción de características y la clasificación de forma local en el dispositivo cliente, y, por consiguiente, limitar la transmisión hacia la nube a cargas puramente textuales.
- Desarrollar la arquitectura de software de una extensión para navegadores basados en la tecnología Chromium, articulada con un motor de ejecución en segundo plano (ILSA).

- Evaluar el clasificador con un script local, seña por seña, según la Sección 7.2.1. Evaluar la fidelidad semántica de los dos canales según la Sección 3.3, aparte de la comprensión del avatar. Medir la latencia por etapas, según la Sección 3.1.3, y el consumo de hardware.

Capítulo 3

Marco teórico, estado del arte y metodología

3.1. Marco teórico

3.1.1. Componentes articulatorios y estructura fonológica de la LSA

La articulación de la LSA se sostiene sobre la interacción simultánea de dos grandes conjuntos de parámetros lingüísticos:

- **Rasgos manuales:** Comprenden la configuración o forma de la mano, la ubicación o punto de articulación en el espacio corporal, la orientación de la palma y la trayectoria cinemática [2].
- **Rasgos no manuales:** Incluyen la gesticulación facial (movimiento de cejas, boca y mirada), la inclinación del torso y la postura del cuello [2]. Estos elementos cumplen un rol determinante para matizar la intensidad de una acción, marcar interrogaciones o delimitar la polaridad en enunciados complejos.

Desde la perspectiva de la fonología gestual, la ejecución temporal de toda seña o glosa se estructura en tres fases secuenciales delimitadas en la literatura lingüística [3]:

- **Preparación (*Movement*):** Transición cinemática desde la postura de reposo o neutra hasta el punto de articulación inicial.
- **Núcleo (*Hold/Stroke*):** Fase central en la cual la configuración manual, la localización espacial y el movimiento otorgan el valor fonológico distintivo a la glosa.
- **Retracción (*Retraction*):** Retorno cinemático desde el término de la seña hacia la postura neutra de reposo.

3.1.2. Estructura lingüística de la LSA y orden manual del prototipo

La LSA y el español rioplatense no comparten la misma gramática. Las afirmaciones generales sobre la lengua y la regla de trabajo de este prototipo van separadas.

Sobre la LSA, la bibliografía de referencia sostiene tres puntos. La organización *Topic-Comment* presenta un tema y después lo que se dice de ese tema [2]. Esa organización no equivale, por sí sola, a un orden fijo de todas las palabras. La lengua no usa los artículos, las preposiciones gramaticales abstractas ni los verbos copulativos del español [2, 4].

Este prototipo no implementa esa gramática completa. El clasificador no toma la boca, las cejas ni la cabeza como rasgos. Una pregunta, una negación o un tiempo solo se recuperan si la persona los produce con una seña manual. La ecuación siguiente es la regla simplificada con la que el prototipo ordena las glosas, no una descripción de toda la LSA:

$$\begin{aligned}\text{Glosas del prototipo} &\rightarrow \text{Tiempo} \rightarrow \text{Lugar} \rightarrow \text{Sujeto} \\ &\rightarrow \text{Objeto} \rightarrow \text{Verbo} \rightarrow \text{Negación o pregunta}\end{aligned} \tag{3.1}$$

Debido a estas divergencias morfosintácticas, la equivalencia entre ambas lenguas no admite un mapeo léxico directo o traducción término a término. Quedan fuera la pregunta o la negación hechas solo con la cara, y cualquier significado que no tenga una seña manual en este recorte.

3.1.3. Sistemas de tiempo real y tiempo pseudo-real

El tiempo pseudo-real, en este prototipo, es una suma de esperas distintas. Cada una se cuenta desde un evento propio, y el servidor no las mezcla.

- **Pausa.** Empieza cuando las manos salen del cuadro y termina a los 4 segundos si no vuelven. Cierra la frase. No es tiempo del modelo.
- **Cola.** Empieza cuando la solicitud llega al servidor y termina cuando el modelo la toma. El servidor informa ese tiempo aparte de la inferencia.
- **Inferencia.** Empieza cuando el modelo toma la solicitud y termina cuando devuelve el texto. Es el único tramo al que el requerimiento RNF-01 exige un máximo de 2 segundos en banda ancha.
- **Transporte.** Empieza cuando el cliente envía el pedido y termina cuando el texto vuelve. Cubre la ida y la vuelta por la red. El servidor no lo mezcla con la cola ni con la inferencia.
- **Entrega.** Empieza cuando el texto vuelve al cliente. En el canal directo incluye el subtítulo y la voz. En el canal inverso incluye la descarga de los movimientos, el dibujo y la salida del video ya compuesto hacia la llamada.

El ciclo que espera el interlocutor suma pausa, transporte, cola, inferencia y entrega. La meta de 2 segundos cubre solo la inferencia. Este capítulo define los tramos; el procedimiento que los mide está en la Sección 7.3.1.

3.1.4. Visión computacional y reconocimiento basado en esqueletos

El reconocimiento de señas a partir de flujo de video crudo opera, en cada fotograma, sobre millones de valores cromáticos. Un cuadro de 1920×1080 píxeles en espacio RGB contiene:

$$1920 \times 1080 \times 3 = 6\,220\,800 \tag{3.2}$$

escalares. Esta representación resulta altamente sensible a las variaciones de iluminación, al fondo y a la vestimenta del usuario. El reconocimiento basado en esqueletos reemplaza esta representación densa mediante la clasificación de un conjunto reducido de coordenadas articulares, denominadas *landmarks*.

Para la estimación postural se utiliza *MediaPipe Holistic*, una arquitectura de percepción distribuida para dispositivos de borde [5, 6]. Este marco no constituye un detector único, sino una serie de modelos preentrenados encadenados, que se ejecuta de forma local en el cliente sin transmitir la imagen cruda. El primer modelo, *BlazePose*, localiza al sujeto y estima 33 puntos de pose corporal [7]. Dichos puntos abarcan la estructura facial gruesa, el torso y las extremidades. A partir de las coordenadas de las muñecas, el pipeline delimita una región de interés (*ROI*) para cada mano, donde el modelo específico de manos (*hand*) infiere 21 puntos tridimensionales por mano (la muñeca y cuatro articulaciones por dedo) [8].

Asimismo, los datos cinemáticos de referencia gestual se archivan en un formato binario especializado con extensión `.pose` [9]. La especificación del formato y su esquema de serialización provienen de la biblioteca *pose-format*, desarrollada para el almacenamiento y manipulación de datos posturales en lenguas de señas, la cual organiza los puntos en un encabezado de metadatos globales y un tensor multidimensional de coordenadas espacializadas.

3.1.5. Modelado de secuencias temporales y atención

La clasificación gestual requiere procesar secuencias temporales de coordenadas articulares para asignar una clase o glosa. Un modelo recurrente, por ejemplo una red LSTM, condensa la historia en un estado de tamaño fijo. Ese estado puede diluir los instantes iniciales del gesto al llegar a la pose final.

El mecanismo de atención permite que cualquier instante consulte otros instantes de la secuencia en una operación matricial, sin pasar por un único estado acumulado [10]. Eso no elimina todas las pérdidas: el *softmax* concentra el peso y la dimensión de las claves es finita. Para cada entrada, el modelo proyecta tres vectores: consulta (Q), clave (K) y valor (V). La respuesta de atención se calcula mediante la relación:

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{3.3}$$

Los símbolos de esa ecuación se leen así:

- k es la dimensión de la entrada de esa capa lineal.
- d es la dimensión de la salida.
- r es la dimensión intermedia del adaptador, con $r \ll \text{mín}(d, k)$.
- α es un factor de escala constante. El cociente α/r regula la magnitud de la actualización.
- $\mathbf{W}_0$ son los pesos preentrenados, que permanecen fijos.
- $\mathbf{B}$ y $\mathbf{A}$ son las únicas matrices que se entrenan.

La figura dibuja el caso cuadrado $k = d$: $\mathbf{B}$ mide $d \times r$ y $\mathbf{A}$ mide $r \times d$. El producto BA recupera el tamaño de ΔW.

3.1.6. Modelos de lenguaje pequeños y adaptación *LoRA*

Los modelos de lenguaje pequeños (SLM, por sus siglas en inglés) poseen un número reducido de parámetros en comparación con las arquitecturas de gran escala. Su función principal consiste en tomar una secuencia de texto de entrada para predecir la continuación lógica del mensaje. En este prototipo, los SLM asumen la traducción semántica bidireccional. Esta dimensión reducida facilita la etapa de ajuste fino, además de permitir la ejecución posterior del traductor, previamente cuantizado, directamente en la CPU.

El procesamiento de información requiere la conversión del texto en tokens, los cuales constituyen los componentes básicos del vocabulario del tokenizador. Cabe destacar que un token no equivale necesariamente a una palabra completa, sino que puede abarcar caracteres individuales, subpalabras o palabras enteras. Estos segmentos representan la unidad fundamental mediante la cual la arquitectura divide la entrada y cuantifica la salida generada.

La adaptación de estos modelos se ejecuta mediante la técnica de Adaptación de Bajo Rango (*LoRA*) [11], la cual congela la matriz de pesos preentrenados $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$ e introduce dos matrices entrenables de bajo rango $\mathbf{B} \in \mathbb{R}^{d\times r}$ y $\mathbf{A} \in \mathbb{R}^{r\times k}$, con rango $r \ll \text{mín}(d, k)$. La actualización lineal adopta la forma:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}(\mathbf{BA}) \tag{3.4}$$

donde α actúa como un factor de escala constante. La figura dibuja el caso cuadrado $k = d$: $\mathbf{B}$ mide $d \times r$ y $\mathbf{A}$ mide $r \times d$. El producto BA recupera el tamaño de ΔW.

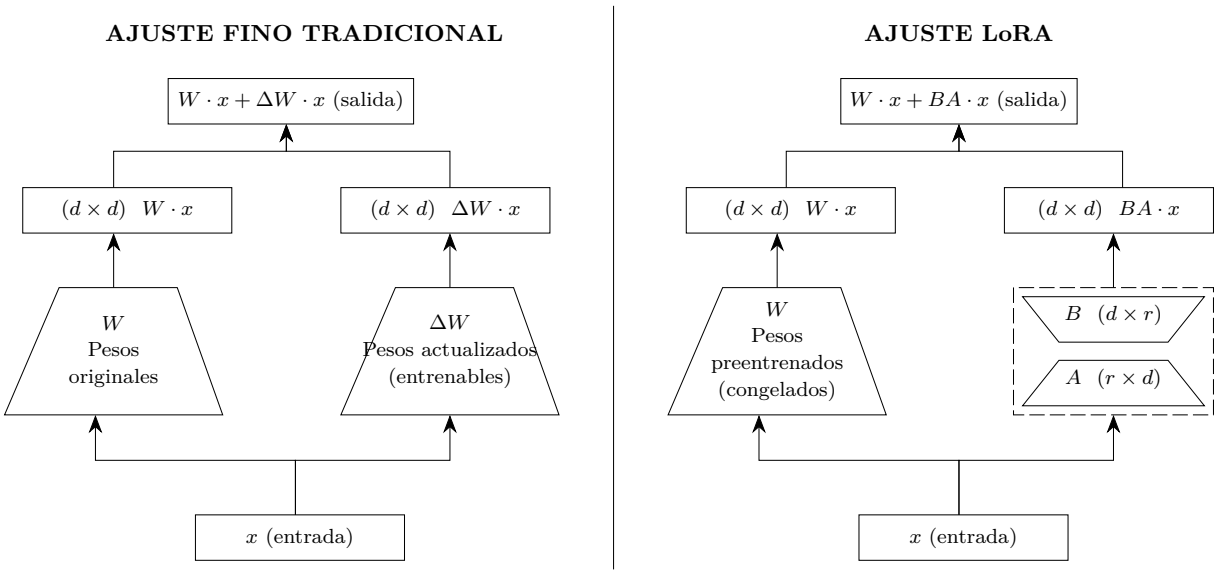


Figura 3.1: Comparación algebraica del ajuste. A la izquierda, un ajuste completo se escribe como $W + \Delta W$. En la práctica se actualizan los pesos originales; no se conserva una base congelada con una rama aparte. A la derecha, LoRA descompone ΔW en B y A. El dibujo muestra el caso $k = d$ y escribe BA; la ecuación 3.4 incluye el factor α/r.

Tras el ajuste, los adaptadores se fusionan con la base. Ese resultado sigue en 16 bits. Hay que distinguir tres cosas. El formato de archivo es GGUF. La configuración de cuantización es el preset `Q4_K_M` de `llama.cpp` [12]. La ecuación de abajo es una reconstrucción simplificada de un peso dentro de un bloque.

El conversor parte cada tensor en superbloques de 256 pesos y, dentro, en bloques más cortos. Cada bloque guarda enteros de 4 bits junto con una escala s y un mínimo m:

$$\hat{w} = m + s\,q \tag{3.5}$$

donde q es el entero de ese peso. En la tabla de cuantización de `llama.cpp`, medida sobre Llama 3.1 8B, `Q4_K_M` queda en 4,8944 bits por peso, cerca de 5 frente a 16 [12]. Esa fila es el archivo completo dividido por los parámetros de ese modelo. No separa la cabecera, el vocabulario ni los tensores de otra precisión, y no se traslada tal cual a los modelos de este trabajo. No es una medición de calidad sobre las oraciones. La mayor parte de los tensores queda en `Q4_K`. El preset reserva `Q6_K` para `v_proj` y para `down_proj`. Las normalizaciones permanecen en 32 bits. Con ese tamaño de archivo, el traductor de las pruebas corre en CPU.

3.1.7. Estándares de captura web y arquitecturas de extensión web

La captura de video en el navegador se apoya en dos especificaciones. *Media Capture and Streams* define `getUserMedia()` [13]. Esa llamada entrega pistas de un dispositivo, por ejemplo la cámara. La misma especificación permite varias pistas asociadas a una misma fuente.

En las pruebas de captura de este trabajo, en Windows con Chrome y Google Meet, la apertura directa de la cámara mediante `getUserMedia()` devolvió la excepción `NotReadableError` cuando la videoconferencia ya tenía el dispositivo. Esa observación no se presenta como una propiedad de todo Windows con Meet. Por ese motivo, la arquitectura de ILSA omite una segunda apertura de la cámara y reutiliza el objeto `MediaStream` de la reunión. Al desactivar la extensión, el sistema conserva los flujos activos y no invoca `stop()` sobre sus pistas.

El lienzo arma la imagen que sale a la llamada. Los fotogramas se leen de ese mismo flujo y el dibujo agrega el subtítulo o el esqueleto. `canvas.captureStream()`, de *Media Capture from DOM Elements* [14], construye un `MediaStream` a partir del lienzo donde esos dibujos ya están.

En el ámbito de las extensiones web bajo el estándar Manifest V3 [15], la Política de Seguridad de Contenido (CSP) prohíbe la ejecución de código dinámico en scripts de fondo. Para ejecutar los clasificadores y estimadores de visión computacional se implementa una arquitectura desacoplada en dos entornos aislados:

- **Documento en segundo plano (*offscreen*):** Entorno habilitado por la API `chrome.offscreen` [16] que provee contexto de DOM completo para coordinar la mensajería y gestionar lienzos desacoplados.
- **Entorno aislado (*sandbox*):** Entorno habilitado por el manifiesto [17] con CSP relajada que autoriza la instanciación de trabajadores web (*Web Workers*) dedicados al cómputo intensivo de inferencia.

3.2. Estado del arte

La comparación usa cuatro ejes: lengua, tipo de entrada, sentido de la traducción e integración en una llamada. El prototipo reconoce señas aisladas de LSA, a partir de los

puntos de una cámara web, en los dos sentidos, dentro de Google Meet. No reconoce un discurso continuo sin corte entre señas.

Cuadro 3.1: Sistemas de referencia según lengua, entrada, sentido e integración.

Sistema	**Lengua y entrada**	**Sentido**	**Integración**
Señario de la CAS [18]	LSA, material pedagógico	Consulta estática.	Fuera de la videollamada.
LSApp [19]	LSA, ítem de diccionario o deletreo	Hacia la animación, en un solo sentido.	Aplicación móvil, sin reconocimiento.
Blessit y Línea 144 [20, 21]	LSA, video con una persona	Interpretación humana.	Videollamada sujeta al turno del operador.
LSA64 y redes recurrentes posteriores [22, 23]	LSA, video con guantes de colores	Reconocimiento de señas aisladas.	Laboratorio. Los guantes impiden el paso directo a una cámara web.
SignAll [24]	ASL, visión	Hacia inglés escrito.	Sin devolución en señas.
WLASL [25]	ASL, video	Reconocimiento léxico.	Corpus. No traduce oraciones ni trabaja en LSA.
Hand Talk [26]	Libras y ASL, texto	Hacia un avatar esquelético.	Sin reconocimiento y sin LSA.
HETAH [27]	Español, oración escrita	Hacia síntesis gestual.	Sin reconocimiento de LSA dentro de una llamada.

Cada límite de la tabla está en la fuente citada: el diccionario no reconoce, LSA64 filma con guantes, SignAll no devuelve señas, WLASL no sale de la ASL léxica, y Hand Talk y HETAH parten del texto. Ninguna de esas fuentes describe el recorte de este prototipo, que es el reconocimiento aislado dentro de Meet.

3.3. Metodología de desarrollo

El proyecto se estructuró bajo un modelo de ciclo de vida iterativo e incremental [28], compuesto por 6 fases principales:

1. **Curación de datos y formalización léxica:** Delimitación de 100 señas de LSA en colaboración con un informante calificado; diseño del entorno de grabación; captura de clips en resolución 1080p; y generación del dataset de coordenadas vectoriales normalizadas.

2. **Entrenamiento y diagnóstico del clasificador gestual:** Prueba de ventanas de 8, 12, 16 y 32 fotogramas; búsqueda de hiperparámetros con Optuna [29]; diagnóstico de confusiones con el AUC de Mann-Whitney; y evaluación en cámara del checkpoint de 100 clases, con diez tomas por seña, dos condiciones de iluminación, y persona y encuadre fijos.

3. **Desarrollo de los módulos semánticos bidireccionales y diseño de despliegue:** Diseño de prompts; ajuste con LoRA sobre Qwen2.5 [30], SmolLM2 [31] y Llama 3.2 1B Instruct [32], en el orden manual de la Sección 3.1.2; cuantización GGUF [12]; y evaluación de fidelidad según la Sección 3.3.

4. **Integración y renderizado gestual:** Acoplamiento del pipeline en el motor local ILSA mediante patrones de diseño desacoplados; comunicación con el servidor semántico; e intercepción de video en Google Meet mediante Canvas sintético y extensión Manifest V3 [15].

5. **Avatar e integración visual inversa:** Desarrollo del controlador del avatar para la representación esquelética de las glosas del canal inverso; implementación del subsistema de contingencia para deletreo manual dactilológico; e integración en la extensión.

6. **Fase de pruebas:** Medición de las etapas de la Sección 3.1.3, con el procedimiento de la Sección 7.3.1.

Fidelidad semántica La fidelidad semántica, en este trabajo, es la conservación de la proposición dentro de los mensajes que el prototipo admite: quién interviene, de qué se habla, en qué tiempo, si hay negación y si hay pregunta. La evaluación automática compara la salida con una referencia. Se registran cinco métricas porque cada una mira un fallo distinto. BLEU y ROUGE-L castigan una paráfrasis correcta y pueden premiar un texto que repite palabras pero cambia la persona, el tiempo, la negación o la pregunta. METEOR afloja esa exigencia cuando coincide la raíz. `bertscore_f1` acepta palabras distintas si sus vectores quedan cerca. `comet_score` mira también la entrada, no solo la referencia.

- **BLEU** [33]. Precisión de n-gramas: qué porción de las secuencias de la salida ya está en la referencia. Una salida corta recibe una penalización. No premia, con el mismo peso, las palabras de la referencia que la salida omitió.

- **ROUGE-L** [34]. La subsecuencia común más larga que respeta el orden. Dos textos pueden compartir palabras y aun así bajar si esas palabras no aparecen en el mismo orden.

- **METEOR** [35]. Alinea palabras sueltas. Cuenta la coincidencia exacta, la raíz y, si hay un léxico de sinónimos, el sinónimo. Junta precisión y recall, y baja si el texto queda partido en muchos fragmentos.

- `BERT Score` [36]. Compara el vector de cada token de la salida con los de la referencia, mediante un modelo de lenguaje preentrenado. El F1 de esa alineación sube cuando el sentido está cerca aunque las palabras no coincidan.

- `Comet Score` [37]. Un modelo entrenado para aproximar juicios humanos de traducción. Usa la entrada, la salida y la referencia. Estima si el sentido de la entrada llegó a la salida.

Un puntaje alto de solapamiento no equivale a comprensión de LSA ni a validación del avatar. La configuración de cada métrica —modelo de `BERT Score`, versión de `Comet Score`, normalización, agregación y tratamiento de referencias múltiples— se informa junto con los resultados, en la Sección 7.2.2.

3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial

Herramientas empleadas. Durante las fases de desarrollo y redacción se utilizaron dos herramientas auxiliares:

- **Cursor:** Entorno de desarrollo integrado asistido por inteligencia artificial, utilizado para la navegación del repositorio, autocompletado sintáctico y asistencia en la edición de código en Python y JavaScript.
- **Gemini (Google):** Modelo de lenguaje utilizado como soporte de consulta técnica, facilitador en la asimilación de conceptos teóricos y asistencia en la revisión del documento. También redactó borradores de pares español–glosas para el corpus semántico, a partir de un prompt que fijaba el orden manual, el vocabulario y los fenómenos a cubrir. Cada par se leyó y se corrigió antes de entrar al archivo de entrenamiento.

Alcance y propósito de su utilización. La intervención de dichas herramientas fue con el propósito de:

- **Consultas técnicas y conceptuales:** Asistencia en la comprensión de aspectos específicos relativos a la cuantización GGUF Q4_K_M con `llama.cpp`, fundamentos de LoRA y esquemas de comunicación asíncrona en extensiones Manifest V3.
- **Asistencia en el desarrollo de software:** Sugerencias de refactorización de código y resolución de incompatibilidades sintácticas en el entorno Chrome Extensions.
- **Revisión de formato:** Corrección gramatical y adecuación de estilo académico del informe.

Límites y responsabilidad de autoría. Ninguna herramienta generativa reemplazó la labor intelectual, analítica ni experimental del proyecto. La concepción de la arquitectura híbrida, la recolección y curación del corpus de 100 señas, el diseño e iteración de las redes neuronales, el ajuste de los modelos de lenguaje y el análisis de resultados constituyen desarrollo de los autores, quienes asumen la responsabilidad técnica y académica por la totalidad del contenido. Los pares del corpus semántico se redactaron con asistencia de Gemini bajo un prompt definido por los autores. Ellos revisaron y aceptaron cada ejemplo antes de entrenar.

3.4. Herramientas y tecnologías empleadas

Cuadro 3.2: Conjunto de tecnologías del sistema integrado

Categoría	**Herramienta**	**Rol en el sistema**
Entorno y Lenguajes	Python 3.11, JavaScript	Lógica de backend, motor local, servidor semántico y extensión web.
Visión Computacional	MediaPipe Holistic, OpenCV	Extracción de coordenadas articulares corporales y manipulación de video.
Aprendizaje Profundo	PyTorch 2.6	Inferencia local del clasificador gestual.
Modelos de Lenguaje	Transformers, PEFT, Unsloth	Fine-tuning eficiente con adaptadores LoRA para traducción directa e inversa.
Inferencia de Lenguaje	`llama.cpp`, GGUF	Inferencia del traductor en CPU.
Comunicación y Backend	FastAPI, Uvicorn	Servidor de interfaz local (ILSA) y microservicio semántico desacoplado.
Servicio en la nube	Google Cloud Run	Instancia del servicio semántico empleada en las pruebas experimentales.
Integración Web	Chrome Extensions (MV3)	Intercepción de media mediante Canvas `captureStream()` en Google Meet.
Transcripción	Vosk	Transcripción local de audio a texto en el cliente.
Síntesis de Salida	Síntesis del navegador y avatar	Generación de voz sintética y dibujo esquelético del avatar gestual.
Avatar	PoseFormat	Conversión de metadatos y almacenamiento en formato binario `.pose`.

Capítulo 4

Análisis de requerimientos

4.1. Requerimientos funcionales

- **RF-01 — Captura y extracción local de landmarks:** Capturar el video del usuario señante en el cliente y extraer los 33 puntos de pose y los 21 de cada mano, en tres coordenadas. Cada punto resta el centro de los hombros y queda dividido por la distancia entre hombros.
- **RF-02 — Clasificación gestual temporal local:** Clasificar localmente secuencias fijas de coordenadas en una de las 100 clases de LSA soportadas y utilizar arquitecturas de aprendizaje profundo para reportar la glosa asignada y su confianza.
- **RF-03 — Traducción semántica directa remota (LSA → Español):** Enviar a la nube el texto de glosas de un enunciado y recibir una oración en español rioplatense. Una glosa léxica sola, un deletreo de letras o una seguidilla de dígitos se resuelven en ILSA y no salen a la nube.
- **RF-04 — Emisión hacia el oyente:** Con Subtítulo, mostrar la oración en el video. Con Audio o Ambos, reproducir la voz para quien señа y para el resto de los participantes de la llamada. En Ambos, el subtítulo y la voz salen juntos.
- **RF-05 — Captura del turno del interlocutor oyente:** Capturar la voz o texto del oyente en la videollamada durante su turno de intervención con el fin de ser procesada para su uso en el canal de retorno bidireccional.
- **RF-06 — Traducción semántica inversa remota (Español → LSA):** Enviar el texto en español al servicio semántico y recibir glosas en el orden manual de la Sección 3.1.2.
- **RF-07 — Representación visual mediante un avatar:** Dibujar el clip de cada glosa que tenga archivo, hasta 24 archivos por oración. Si falta el archivo, deletrear letra por letra. El deletreo no asegura la comprensión del significado. Lo que supere el tope queda fuera del dibujo y se conserva en texto. Si el servidor de movimientos no responde, el clip queda vacío y las glosas siguen en texto. Encolar cada oración: la cola admite hasta 20 animaciones, y una oración nueva espera su turno sin interrumpir la animación en curso.
- **RF-08 — Integración con Google Meet:** Inyectar el video procesado y los canales de comunicación dentro de la pestaña de videollamada.

- **RF-09 — Arranque del panel:** Al Encender, Reiniciar o Apagar, volver a la pantalla de carga. Abrir el panel principal solo si responden el traductor semántico y el servidor de poses. Si alguno falla, dejar el botón Reintentar y no abrir el panel.

4.2. Requerimientos no funcionales

- **RNF-01 — Latencia y tiempo de respuesta:** La pausa, la cola, la inferencia, el transporte y la entrega se cuentan por separado, como fija la Sección 3.1.3. La meta de 2 segundos rige solo para la inferencia, con el servicio ya activo. No cubre la cola, el transporte, el arranque en frío ni la pausa de 4 segundos. El cumplimiento de esa meta se informa en la Sección 7.3.1, con inferencia, cola y red en campos distintos.
- **RNF-02 — Datos que recibe cada componente:** Meet recibe el video ya compuesto (cámara, subtítulo y, en modo oyente, el esqueleto) y el audio de la llamada, incluida la voz sintética cuando la salida es Audio o Ambos. ILSA, en la misma computadora, recibe los puntos articulares y el audio local para transcribir. El traductor en la nube recibe solo texto: glosas u oración. El servidor de movimientos recibe el nombre de una glosa o las letras de un deletreo, y devuelve archivos. La prohibición de enviar imagen o video crudo rige para el traductor y para el servidor de movimientos. El tratamiento dentro de este alcance es ese reparto: la imagen de la cámara sale hacia Meet porque es la llamada, y no sale hacia los dos servicios de la nube.
- **RNF-03 — Robustez en reconocimiento en vivo:** El clasificador debe superar el 75 % de acierto en luz doméstica. Ese valor es una meta de reconocimiento.
- **RNF-04 — Desacoplamiento y escalabilidad de los servicios en la nube:** La arquitectura del backend semántico y del servidor de archivos .pose deben estar concebidas para operar de forma desacoplada y elástica, lo que permita sus despliegues en infraestructuras con auto-escalado horizontal a través de interfaces REST tipadas.
- **RNF-05 — Usabilidad y no invasividad:** La interfaz sobre la videollamada debe ofrecer indicadores visuales de estado que no ocluyan la visibilidad del interlocutor para mitigar la brecha comunicacional receptiva y expresiva.

4.3. Restricciones

- **RC-01 — Alcance léxico:** El vocabulario reconocido por el clasificador está acotado a 100 señas validadas por profesionales de LSA, enmarcadas en las pautas pedagógicas y lingüísticas vigentes.
- **RC-02 — Despliegue en la nube del prototipo:** Las pruebas no despliegan la arquitectura elástica del servicio semántico o del servicio de archivos .pose. La validación usa Cloud Run, de Google Cloud.
- **RC-03 — Compatibilidad de plataforma cliente:** El empaquetado del motor local (ILSA) y la extensión estan diseñados para el sistema operativo Windows y navegadores basados en Chromium bajo las especificaciones de seguridad vigentes.

- **RC-04 — Dependencia de visión:** El clasificador depende del correcto funcionamiento de MediaPipe Holistic para la extracción de landmarks.
- **RC-05 — Recursos del cliente objetivo:** El componente local del prototipo debe ser capaz de operar en laptops estándar (procesadores de 4 núcleos y 8 GB de memoria RAM, sin placa gráfica dedicada).

4.3.1. Especificación del entorno de ejecución cliente

El cuadro siguiente propone un equipo mínimo y uno con más margen. No informa una prueba sobre cada modelo de procesador ni sobre cada navegador de la lista. El nombre comercial de una familia cubre piezas distintas: el criterio operativo es el número de núcleos, el soporte AVX2 y el piso de reloj, no la etiqueta Core i3 o Ryzen 3.

El mínimo se elige para que convivan tres cargas: la videollamada, la extracción de puntos en el navegador y la clasificación de ILSA en CPU [15]. De ahí salen cuatro núcleos, 8 GB de RAM, gráficos con WebGL para el lienzo, disco de estado sólido, cámara de 720p y banda ancha. Chrome 115 o posterior es la versión en la que se comprobó Manifest V3. Windows 10/11 de 64 bits es el sistema del empaquetado de ILSA.

La fila recomendada es una propuesta de holgura: más núcleos, más memoria y cámara de 1080p. No es una configuración medida. Edge, Brave, Opera y Vivaldi comparten motor Chromium, y por eso se proponen. Este trabajo no informa una pasada en cada uno: las políticas y las APIs pueden diferir. La comparación entre la máquina de desarrollo y la máquina objetivo queda en la Sección 7.3.4. Esa sección describe el comportamiento observado; no informa porcentajes de CPU ni de memoria. El cuadro de mínimos de esta sección es una propuesta de entorno, no una batería de pruebas por modelo de procesador.

Cuadro 4.1: Requisitos de hardware y software del dispositivo cliente.

Componente	Requisitos mínimos (Dispositivo base)	Requisitos recomendados
Procesador (CPU)	Intel Core i7-10510U / i5 (10ª gen.) o AMD Ryzen 3 (serie 3000) (4 núcleos / 8 hilos, base $\geq$ 1,8 GHz, AVX2).	Intel Core i5 (11ª gen.+) / AMD Ryzen 5 (serie 4000+) (6 núcleos / 12 hilos).
Memoria RAM	8 GB DDR4 ($\geq$ 2,5 GB libres para el navegador y entorno de ejecución).	16 GB DDR4 / DDR5.
Gráficos (GPU)	Gráficos integrados Intel UHD Graphics 620 / 630 o AMD Radeon Vega 3 con soporte WebGL 2.0.	Intel Iris Xe / AMD Radeon 600M+ o GPU dedicada básica.
Almacenamiento	Disco de estado sólido (SSD) con 2 GB de espacio libre disponible.	SSD NVMe con $\geq$ 5 GB libres.
Cámara web	Resolución 720p (1280×720) a 24/30 FPS.	Resolución 1080p a 30 FPS.
Conectividad	Conexión de banda ancha ($\geq$ 10 Mbps bajada / $\geq$ 3 Mbps subida, latencia $<$ 80 ms).	Conexión por fibra óptica ($\geq$ 50 Mbps).
Entorno de software	Windows 10/11 (64 bits), Google Chrome 115+ (soporte Manifest V3).	Windows 11 (64 bits), Google Chrome última versión estable.

Condiciones de funcionamiento La CPU debe sostener a la vez la videollamada, MediaPipe en WebAssembly y el clasificador en CPU. El informe no asigna núcleos por carga ni un tiempo de clasificación en milisegundos. Las condiciones que importan, en el mínimo propuesto, son: 4 núcleos con AVX2, 8 GB de RAM con memoria libre para el proceso, WebGL, Windows 10/11 de 64 bits y Chrome 115 o posterior.

4.4. Casos de uso

4.4.1. Casos de uso esenciales en formato breve

- **CU-01 — Conversar con otra persona (Modo Sordo):** El usuario sordo seña frente a la cámara durante la videollamada. El sistema clasifica las señas en el equipo. Una glosa léxica sola, solo dígitos, o un deletreo de dos o más letras sin I ni T se arman en ILSA. El resto consulta el servicio semántico. Con Subtítulo, la oración queda en el video. Con Audio o Ambos, la voz suena para todos los participantes de la llamada.
- **CU-02 — Mano hábil:** ILSA muestra diestro y zurdo, con diestro ya seleccionado. La persona confirma diestro o pasa a zurdo. Con zurdo, el vector se espeja en horizontal y se cruzan las dos manos durante esa sesión.

- **CU-03 — Configurar canal de salida:** En el panel de la extensión, el usuario señante elige la salida en Meet: Subtítulo, Audio o Ambos. Si no toca el control, queda Subtítulo.
- **CU-04 — Traducir español a LSA mediante Avatar (Modo Oyente):** El usuario oyente habla e ILSA transcribe esa voz. El chat es el flujo alternativo: ILSA toma ese texto y no lo transcribe. Parte el texto por espacios. Una sola palabra pasa a mayúsculas y es la glosa, sin traductor. Dos o más palabras van al servicio semántico. El avatar encola hasta 20 animaciones, dibuja hasta 24 clips y deletrea lo que no tiene archivo. El deletreo no asegura la comprensión.
- **CU-05 — Entrar al panel:** Encender, Reiniciar o Apagar vuelven a la pantalla de carga. El panel se abre si responden el traductor semántico y el servidor de poses. Si alguno falla, queda Reintentar.

4.4.2. Diagrama de casos de uso

La Figura 4.1 ubica a los dos actores frente a las cinco funciones del prototipo. El usuario señante opera el canal directo y la configuración de la sesión. El usuario oyente opera el canal inverso. Ambos pueden abrir el panel. El recuadro punteado delimita el sistema: ILSA, la extensión y los servicios remotos que esas funciones consultan. El detalle de precondiciones, pasos y alternativas está en las tablas de la subsección siguiente.

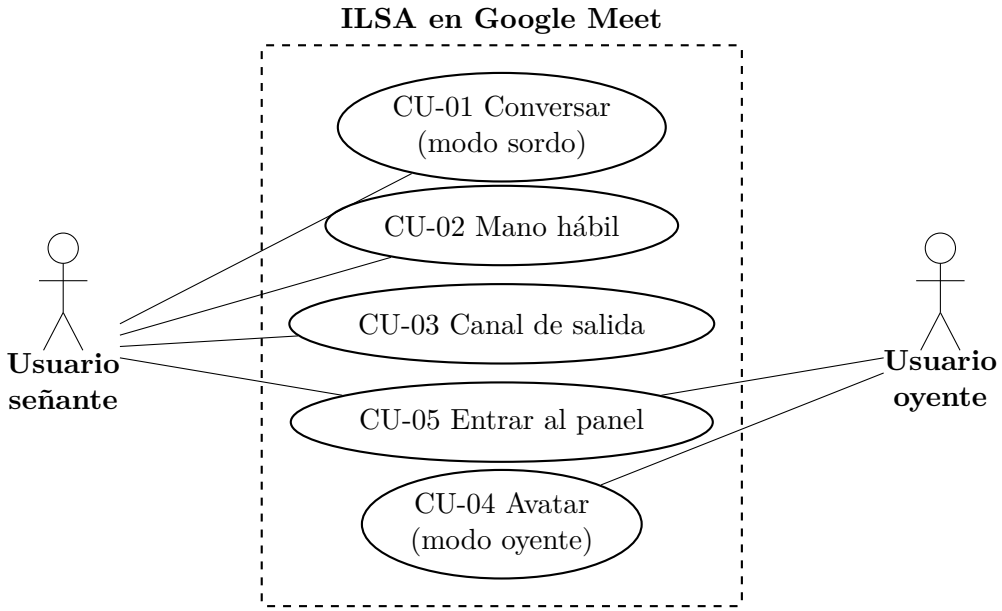


Figura 4.1: Diagrama de casos de uso del prototipo. El señante opera el canal directo y la configuración. El oyente opera el canal inverso. Ambos pueden abrir el panel.

4.4.3. Casos de uso expandidos

<table>
<tr><td>Identificador</td><td colspan="3">CU-01: Conversar con otra persona (Modo Sordo)</td></tr>
<tr><td>Actor principal</td><td colspan="3">Usuario señante</td></tr>
<tr><td>Precondiciones</td><td colspan="3">El usuario está unido a una llamada. ILSA está encendido y el panel principal está abierto.</td></tr>
<tr><td>Postcondiciones</td><td colspan="3">El interlocutor lee el subtítulo en el video de Meet. Con Audio o Ambos, la voz suena en el equipo de quien seña, mediante la síntesis del navegador; esa locución no se mezcla en el <code>MediaStream</code> de la llamada.</td></tr>
<tr><td rowspan="4">Escenario Principal de éxito (flujo básico)</td><td></td><td>Usuario señante</td><td>Sistema</td></tr>
<tr><td>1</td><td>Realiza las señas correspondientes a lo que quiere comunicar.</td><td></td></tr>
<tr><td>2</td><td></td><td>Clasifica la seña. Si la confianza es de al menos 0,75 y el filtro de repetición la admite, el HUD la muestra en cian («En el mensaje: HOLA») y esa glosa queda en la frase.</td></tr>
<tr><td>3</td><td>Si necesita otra seña, vuelve al paso 1. Si no, espera 4 segundos sin manos en el cuadro.</td><td></td></tr>
<tr><td></td><td>4</td><td></td><td>Arma el texto en el equipo si hay una sola glosa léxica, solo dígitos, o solo letras desde dos y sin I ni T. Van al traductor una letra sola, un deletreo con I o T, OJO, VIERNES o AÑOS solas, y las demás frases. Con Subtítulo, deja la oración en el video. Con Audio o Ambos, reproduce la voz en el equipo de quien seña.</td></tr>
<tr><td>Extensiones (flujos alternativos)</td><td colspan="3">1a. Las manos no entran al cuadro: no empieza ninguna seña y no hay plazo que contar.
2a. Confianza bajo 0,75: el HUD muestra la predicción en coral, «Poca confianza: HOLA». Esa predicción no entra en la frase y no se envía. Quien seña puede repetirla antes de los 4 segundos sin manos.
2b. Confianza de al menos 0,75 y repetición no admitida: el HUD muestra gris, «No se agregó: HOLA». La predicción no entra en la frase.
2c. Seña ajena al vocabulario con confianza de al menos 0,75: el HUD la muestra en cian y la glosa incorrecta entra. No hay rechazo automático.
4a. El traductor no responde: el subtítulo queda en el video.</td></tr>
</table>

Identificador	**CU-02: Mano hábil**		
Actor principal	Usuario señante		
Precondiciones	ILSA está en el panel principal. El control de mano muestra diestro seleccionado.		
Postcondiciones	La sesión queda con la mano elegida. Con zurdo, el vector se espeja y se cruzan las manos.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Ve diestro ya seleccionado y lo confirma.	
	2		Conserva la mano derecha. El vector no se espeja.
Extensiones (flujos alternativos)	**1a.** Selecciona zurdo: ILSA espeja el eje horizontal y cruza las dos manos, porque el modelo se entrenó con mano derecha. Conserva esa mano durante la sesión.		
Requisitos especiales	RF-01, RF-02.		

Identificador	**CU-03: Configurar canal de salida del oyente**		
Actor principal	Usuario señante		
Precondiciones	El usuario está unido a una llamada. El panel de la extensión está disponible.		
Postcondiciones	La salida queda en Subtítulo, en voz local, o en ambas. El subtítulo es la vía que llega a Meet. Si no hay elección, queda Subtítulo.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Abre el panel de la extensión.	
	2		Muestra tres opciones bajo «Salida en Meet»: Subtítulo, Audio y Ambos.
	3	Elige una opción.	
	4		Aplica esa salida. Subtítulo deja el texto en el video y no emite voz. Audio o Ambos reproducen la voz en el equipo de quien seña; el resto de los participantes no la oye en la llamada.
Extensiones (flujos alternativos)	**3a.** No hay elección: se conserva Subtítulo. No hay voz. La recuperación es abrir el panel y elegir Audio o Ambos.		
Requisitos especiales	RF-04, RF-08.		

<table>
<tr><td>Identificador</td><td colspan="3">CU-04: Traducir español a LSA mediante Avatar (Modo Oyente)</td></tr>
<tr><td>Actor principal</td><td colspan="3">Usuario oyente</td></tr>
<tr><td>Precondiciones</td><td colspan="3">El usuario está unido a una llamada y la extensión está en modo oyente.</td></tr>
<tr><td>Postcondiciones</td><td colspan="3">El interlocutor ve el esqueleto de las glosas que tienen archivo. Lo que no se dibuja queda en texto.</td></tr>
<tr><td rowspan="5">Escenario Principal de éxito (flujo básico)</td><td></td><td>Usuario oyente</td><td>Sistema</td></tr>
<tr><td>1</td><td>Habla durante la llamada.</td><td></td></tr>
<tr><td>2</td><td></td><td>Transcribe esa voz en la misma computadora.</td></tr>
<tr><td>3</td><td></td><td>Parte el texto por espacios. Una sola palabra pasa a mayúsculas y es la glosa, sin traductor ni diccionario. Dos o más palabras van al servicio semántico.</td></tr>
<tr><td>4</td><td></td><td>Encola la animación, hasta 20, sin cortar la que está en curso. Dibuja hasta 24 clips. Si falta un archivo, deletrea; eso no asegura la comprensión.</td></tr>
<tr><td>Extensiones (flujos alternativos)</td><td colspan="3">1a. Escribe en el chat: ILSA toma ese texto, sin transcribirlo, y sigue en el paso 3. La palabra aislada también llega al paso 4.
3a. Dos o más palabras y el traductor no responde: el cliente no declara un plazo propio y espera la respuesta o el corte del servidor (hasta 300 s en Cloud Run). No hay glosas nuevas y se conserva el video. Una respuesta tardía no se aplica. La recuperación es repetir el turno.
4a. Más de 24 archivos: se dibujan los primeros 24 y el resto queda en texto.
4b. El servidor de poses no responde: misma espera que en 3a. Las glosas quedan en texto y el clip queda vacío. La recuperación es repetir el turno.
4c. ILSA deja de responder: no hay transcripción ni dibujo nuevo. La recuperación es Encender o Reintentar.</td></tr>
<tr><td>Requisitos especiales</td><td colspan="3">RF-05, RF-06, RF-07, RF-08.</td></tr>
</table>

Identificador	**CU-05: Entrar al panel**		
Actor principal	Usuario		
Precondiciones	ILSA está instalado en la computadora.		
Postcondiciones	Si responden el traductor semántico y el servidor de poses, se abre el panel principal.		
Escenario Principal de éxito (flujo básico)		**Usuario**	**Sistema**
	1	Pulsa Encender, Reiniciar o Apagar.	
	2		Vuelve a la pantalla de carga y consulta el traductor semántico y el servidor de poses.
	3		Ambos responden. Abre el panel principal.
Extensiones (flujos alternativos)	**2a.** El traductor o el servidor de poses no responde: el cliente no declara un plazo propio y espera la respuesta o el corte del servidor (hasta 300 s en Cloud Run, plazo que cubre también un arranque en frío). Queda el botón Reintentar y el panel principal no se abre. La recuperación es pulsar Reintentar, que repite la consulta de los dos servicios. **2b.** ILSA no está en ejecución: la consulta no llega a los servicios. La recuperación es Encender.		
Requisitos especiales	RF-09.		

Capítulo 5

Diseño del Sistema

5.1. Arquitectura del sistema

El diseño adopta una arquitectura híbrida desacoplada. La extracción de puntos articulares y la clasificación gestual se ejecutan en el equipo del usuario, mientras que el módulo de traducción semántica y servidor de poses operan como un servicio remoto distribuido.

5.1.1. Justificación de la partición y modelo de privacidad

La extracción de puntos clave y la clasificación de señas se ejecutan de manera local en el dispositivo del usuario. Por esta razón, el servicio de traducción y el servidor de poses nunca reciben la señal de video original ni las coordenadas articulares.

La cámara transfiere la imagen directamente desde el equipo hacia Google Meet para cumplir con la exigencia RNF-02. Así, la videollamada procesa el video compuesto, el cual integra la toma original, los subtítulos y la superposición del esqueleto en el modo oyente.

Debido al consumo de memoria y al costo computacional de los modelos de lenguaje, la traducción se aloja en un servicio remoto. Hacia este componente solo se transmiten cadenas de texto, representadas por la secuencia de glosas o la oración en español. Aunque este flujo no incluye material de video, el texto puede incluir datos personales como nombres o números de documento. Por su parte, el servidor de poses procesa la solicitud a partir del nombre de una glosa o las letras de un deletreo, para luego retornar el archivo correspondiente.

5.1.2. Flujo integral de datos entre componentes

La interacción entre el navegador, el motor local y los servicios remotos se organiza según la estructura representada en la Figura 5.1. El procesamiento local gestiona la captura y clasificación, mientras que la red transmite exclusivamente mensajes en formato JSON.

Para detallar la secuencia de intercambio de mensajes, las Figuras 5.2 y **??** ilustran el comportamiento de los canales directo e inverso respectivamente.

En ese diagrama, ILSA arma el texto en el equipo en tres casos: una sola glosa léxica, un enunciado de solo dígitos, o solo letras de dos en adelante y ninguna es I ni T. Una letra sola, un deletreo que incluye I o T, y las señas OJO, VIERNES o AÑOS cuando están

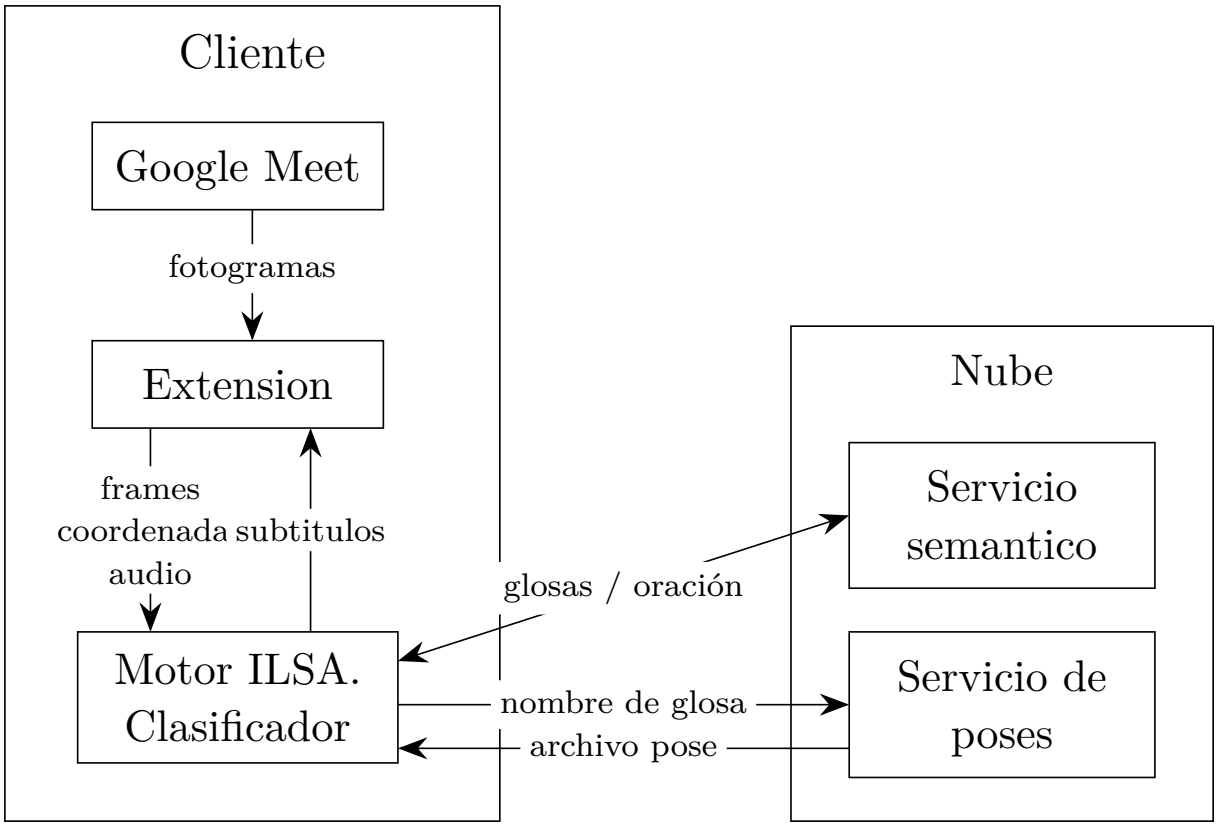


Figura 5.1: Diagrama de la arquitectura general del sistema.

solas van al traductor, igual que una frase mixta. Antes de ese envío, las letras seguidas se juntan y los dígitos seguidos también.

En ese diagrama, ILSA parte el texto por espacios antes de llamar al servidor semántico. Si queda una sola palabra, la pasa a mayúsculas y esa cadena es la glosa: el recuadro «1 palabra» no sale hacia el traductor. Si quedan dos o más palabras, la oración sigue por `POST /hearing`. La decisión no consulta el diccionario de poses. La pose de la glosa, venga de este atajo o del traductor, se pide después, en el recuadro de cada glosa.

5.2. Diseño de contratos de API REST

La comunicación entre el cliente local y los microservicios se rige por contratos JSON estructurados.

5.2.1. API local del motor ILSA

El motor ILSA expone una interfaz HTTP en el bucle local. La extensión web actúa como cliente y el proceso local como servidor. El esquema general de componentes internos se representa en la Figura 5.4.

El Cuadro 5.1 resume las rutas expuestas por el motor local.

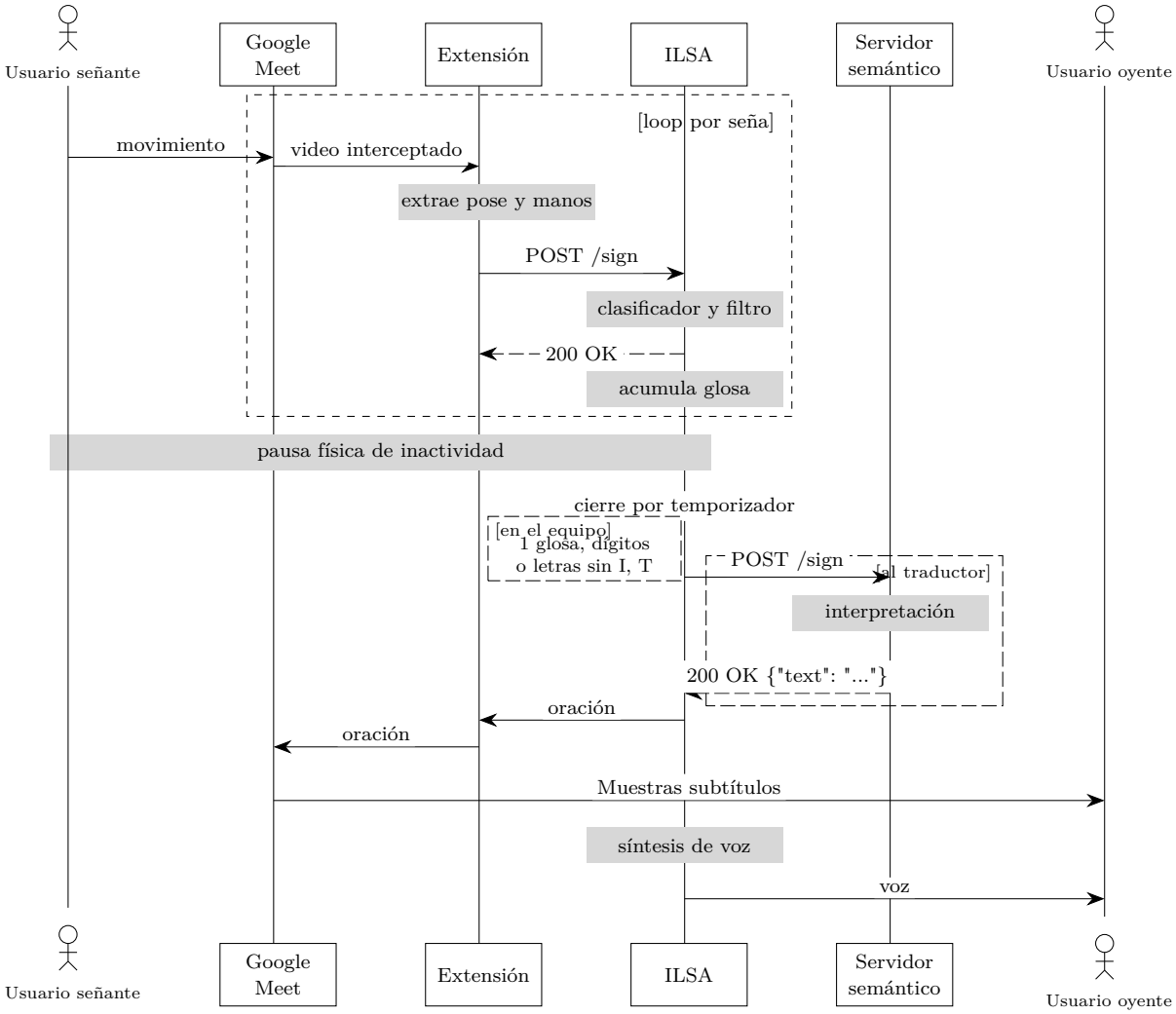


Figura 5.2: Diagrama de secuencia del flujo de traducción de Lengua de Señas a voz y subtítulos.

5.2.2. API del servicio semántico remoto

El microservicio remoto en la nube expone contratos REST sobre HTTPS para resolver la traducción semántica desacoplada. El Cuadro 5.2 resume los endpoints principales.

5.2.3. API del servicio de archivos del avatar

El subsistema de representación gestual de salida consulta un repositorio de archivos cinemáticos binarios. El Cuadro 5.3 detalla las rutas disponibles.

5.3. Diseño de la representación gestual inversa

El canal inverso procesa la lista de glosas devuelta por el traductor e infiere la secuencia cinemática correspondiente. Cada glosa se asocia a un clip de referencias cinemáticas previamente parametrizado.

Cuadro 5.1: Endpoints de la API local del motor ILSA (`http://127.0.0.1:8765`).

Método	**Ruta**	**Contrato**	**Códigos**
GET	`/health`	Disponibilidad del proceso, del modo, del clasificador y del traductor remoto.	200
GET	`/config`	Umbrales de captura del modo en curso.	200, 503
GET	`/state`	Búfer de glosas, último texto y estado del traductor.	200, 503
GET	`/metrics`	Conteos y tiempos de la sesión en curso.	200
POST	`/session`	Apertura de sesión. El cuerpo fija la lateralidad mediante `left_handed`.	200, 503
POST	`/sign`	Ingesta de hasta 60 fotogramas para clasificación en modo señante.	200, 400, 503
POST	`/stt`	Transcripción de audio WAV en Base64 a texto en modo oyente.	200, 400, 502
POST	`/hearing`	Envío de oración y recepción de animación en modo oyente.	200, 400, 503
POST	`/activity`	Reinicio del temporizador de pausa de inactividad ($\sim 4\,\mathrm{s}$).	200, 503
POST	`/utterance/end`	Cierre explícito del enunciado y disparo de traducción.	200, 503
GET	`/mediapipe/{name}`	Despacho de recursos estáticos del estimador visual.	200, 400, 404
GET	`/download/exe`	Entrega del ejecutable del motor local.	200, 404

Cuadro 5.2: Endpoints del servicio semántico remoto.

Método	Ruta	Función
GET	`/health`	Verificación de disponibilidad de los modelos de lenguaje en memoria.
POST	`/sign`	Canal directo: traducción de glosas LSA a oración en español rioplatense.
POST	`/hearing`	Canal inverso: traducción de oración en español a glosas en estructura *Topic-Comment*.

Cuadro 5.3: Endpoints del servicio de archivos del avatar.

Método	Ruta	Función
GET	`/health`	Disponibilidad del proceso de archivos.
GET	`/tposes/{glosa}.pose`	Entrega del flujo binario cinemático correspondiente a la glosa solicitada.
GET	`/fingerspell`	Solicitud de clips del alfabeto manual para deletreo dactilológico.

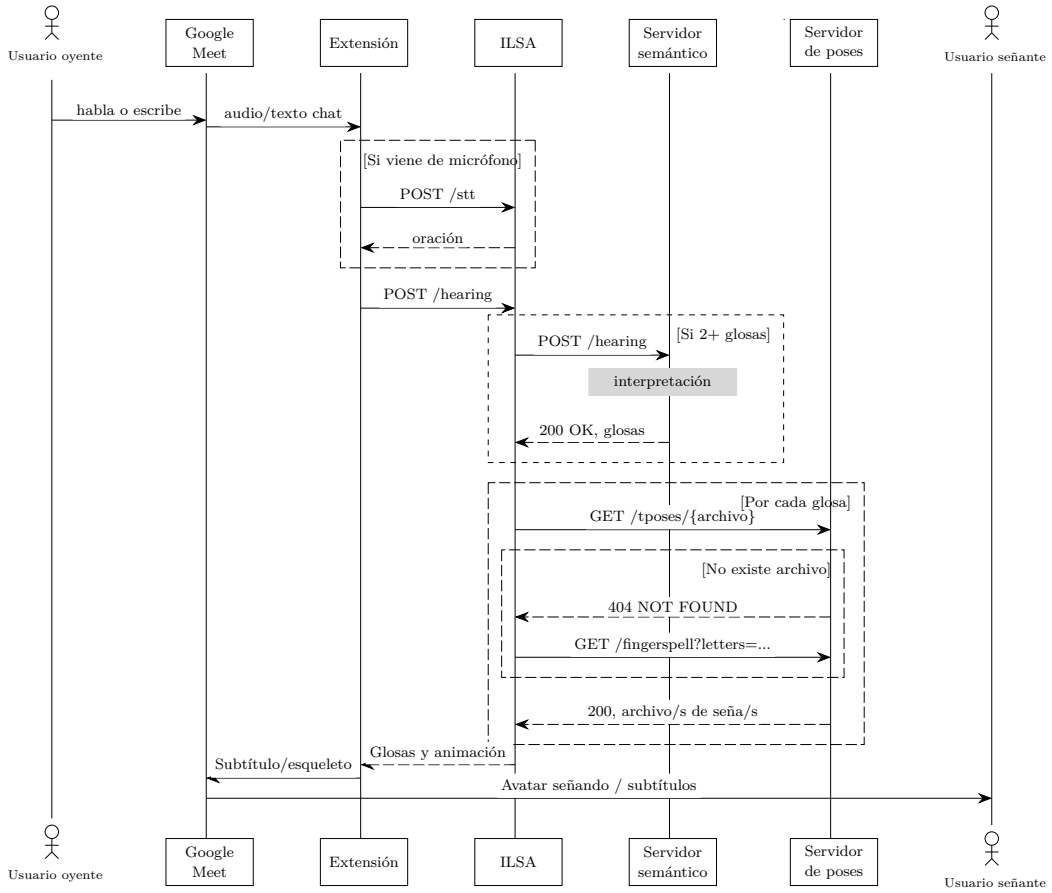


Figura 5.3: Diagrama de secuencia del sistema

5.3.1. Procesamiento geométrico de coordenadas y segmentación temporal

Los clips cinemáticos registran la secuencia de puntos clave extraídos mediante la ejecución previa e independiente de *MediaPipe Holistic* sobre el video de un informante calificado. Las coordenadas planares (x, y) se representan en el espacio discreto de píxeles de la cámara de captura ($x \in [0, W]$, $y \in [0, H]$) y preservan las proporciones nativas. La componente de profundidad z posee carácter relativo con origen en la muñeca y actúa como métrica para resolver oclusiones mutuas. Por consiguiente, la proyección gráfica reproyecta x e y directamente sobre el plano bidimensional.

El archivo binario `.pose` conserva las tres fases fonológicas estándar (preparación, núcleo, retracción). Para aislar el tramo con valor morfosintáctico distintivo sin alterar el archivo original, el sistema registra los marcadores temporales de inicio (t_{inicio}) y fin (t_{fin}) del núcleo en un índice de anotaciones externo. Durante la renderización, la lectura consulta este índice para aislar el segmento relevante.

5.3.2. Preparación, acondicionamiento y ensamblado cinemático

El servicio de poses entrega el archivo `.pose` que corresponde a la glosa pedida. Esa preparación corre en ILSA, después de la descarga. La extensión recibe la lista de fotogramas ya armada y la dibuja; no rehace el empalme.

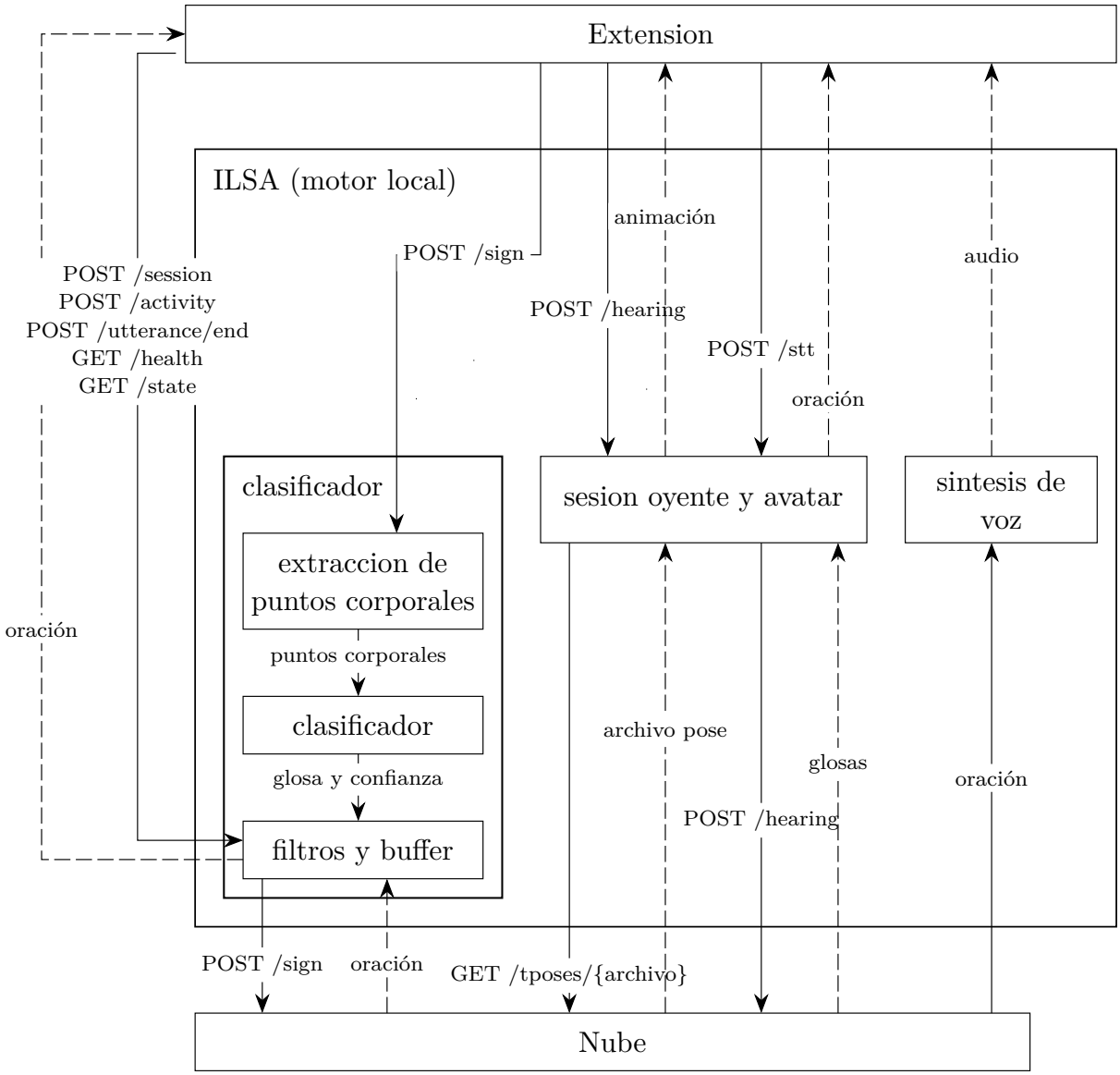


Figura 5.4: Arquitectura detallada del motor local ILSA y su interacción con los servicios en la nube.

ILSA aplica entonces estas operaciones sobre cada archivo recibido:

1. **Normalización temporal:** Conversión de los clips a una cadencia común de 30 fotogramas por segundo.
2. **Aislamiento del núcleo:** Recorte del tramo delimitado por t_{inicio} y t_{fin}.
3. **Interpolación de faltantes:** Relleno de ausencias puntuales mediante interpolación entre fotogramas válidos adyacentes.
4. **Ajuste de extremidad inactiva:** Reposicionamiento de la mano no participante hacia el costado del torso.
5. **Filtro de suavizado:** Aplicación de un promedio exponencial para atenuar la oscilación entre fotogramas.

6. **Ajuste de escala:** Normalización del tamaño corporal en función de una distancia inter-hombros fija.

Antes de la reproducción de los clips, para evitar discontinuidades espaciales abruptas al empalmar el núcleo del gesto k con el núcleo $k+1$, ILSA interpola un *puente sintético* de fotogramas de transición:

$$S(x) = 3x^2 - 2x^3 \quad \text{para } x \in [0,1] \tag{5.1}$$

Este puente ejecuta una traslación suave sobre las coordenadas planares. Se explicita que este procedimiento constituye un ajuste gráfico e interpolatorio, diferendo del fenómeno biomecánico natural de la coarticulación gestual. El Cuadro 5.4 sintetiza las previsiones de diseño ante contingencias en la representación.

Cuadro 5.4: Previsiones del diseño frente a fallos de estimación o léxico.

Situación	**Previsión**
Puntos nulos puntuales en el estimador.	Interpolación lineal a partir de fotogramas válidos vecinos.
Inexistencia de archivo para una glosa.	Activación del subsistema de deletreo manual dactilológico.
Secuencias extensas ($>$ 24 glosas o $>$ 1200 cuadros).	Truncado de la secuencia en el límite máximo tolerable.

5.3.3. Estrategia de contingencia léxica

Ante palabras ausentes en el vocabulario grabado (nombres propios o términos técnicos), el sistema aplica un mecanismo de descomposición dactilológica. La palabra se segmenta en sus letras constituyentes y se sintetiza mediante la secuencia correspondiente de caracteres del alfabeto dactilológico al consultar el endpoint específico `/fingerspell`.

5.4. Diseño de la extensión de videollamada

La integración con la plataforma Google Meet se resuelve mediante una extensión de navegador que intercepta la señal de video y gestiona la superposición de capas gráficas. La Figura 5.5 detalla la distribución funcional de sus componentes.

5.5. Patrones de diseño de software aplicados

El diseño incorpora patrones de software consolidados para organizar las responsabilidades del sistema:

- **Estrategia (*Strategy*):** Aplicado en el módulo de filtrado de repeticiones (`RepeatGate`). Define políticas diferenciadas según la clase léxica (dígitos admiten repetición continua, letras aceptan hasta dos instancias consecutivas y vocabulario general descarta duplicados inmediatos).
- **Fachada (*Facade*):** Aplicado en la API HTTP del motor ILSA local. Expone una interfaz unificada que oculta la complejidad interna de extracción, clasificación y comunicación con la nube.

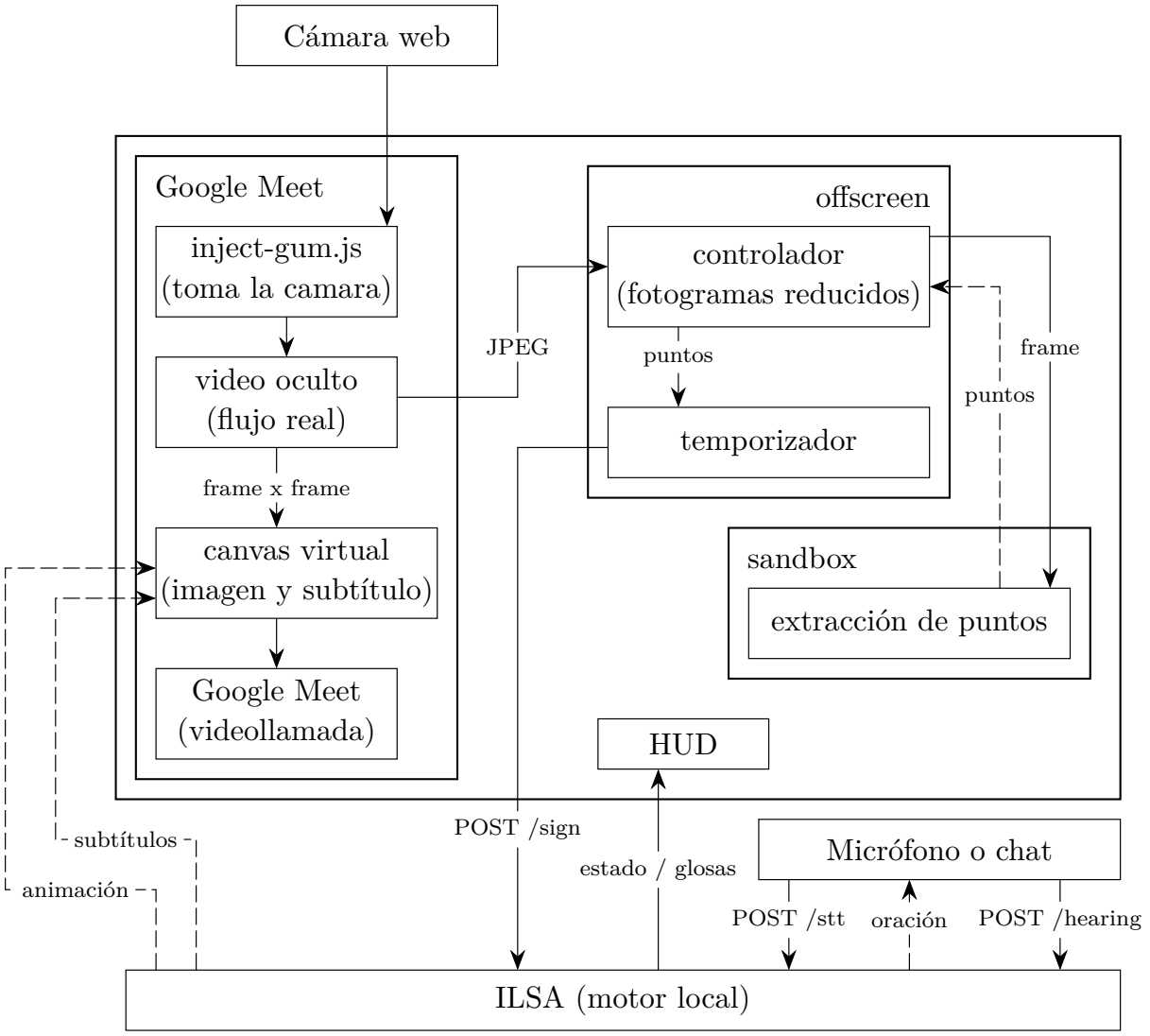


Figura 5.5: Diagrama de arquitectura del flujo de video, procesamiento offscreen e integración con el motor ILSA.

- **Adaptador (*Adapter*):** Implementado en la extensión web y en los conectores de red para transformar los formatos nativos del navegador a los contratos JSON requeridos por los servicios REST.
- **Estado (*State*):** Gestiona la modalidad operativa de la sesión (Señante u Oyente) y altera dinámicamente el comportamiento de los endpoints sin requerir instancias de servicio separadas.

5.6. Diseño conceptual de la arquitectura escalable de producción en la nube

Las pruebas de este trabajo usan una instancia de Cloud Run por servicio. Esta sección no describe esa instancia. Describe la proyección para varias videollamadas a la vez, en coincidencia con la restricción RC-02.

El diagrama, en hoja horizontal, está en el Apéndice C (Figura C.1).

La proyección tiene dos servicios detrás de un mismo ingreso. El traductor genera texto. El servicio de archivos entrega un clip de seña. No comparten réplica ni cola.

5.6.1. Ingreso

Las computadoras cliente salen por HTTPS. El ingreso resuelve el nombre, presenta el certificado TLS y filtra tráfico elemental. Detrás opera un balanceador.

- `POST /sign` y `POST /hearing` van al traductor. El criterio es el menor número de conexiones activas, para no cargar una réplica que ya genera un enunciado largo.
- `GET /tposes/{glosa}.pose`, `GET /fingerspell` y la salud de ese servicio van a las réplicas de archivos.

5.6.2. Traductor

El traductor viaja en una imagen de contenedor. Cada réplica monta los binarios GGUF Q4_K_M desde un volumen de solo lectura, así un arranque nuevo no vuelve a bajar los modelos. Un escalador horizontal mira la cola y el tiempo de generación. Si los turnos simultáneos superan a las réplicas activas, el orquestador abre contenedores nuevos. Cada réplica corre en procesador, sin placa de video. Carga el modelo principal de los dos canales, Qwen2.5 1,5B, con un contexto de 2048 tokens, y deja el respaldo Llama 3.2 1B en disco hasta que ese principal no carga. La estimación de menos de 2 GiB por réplica es la suma de esos dos archivos y de sus cachés de claves y valores, calculada en la Sección 7.3.3. El buffer de activaciones, definido junto a la Ecuación 6.2, queda fuera de esa suma. Los 8 GiB de la Sección 6.6 son la memoria asignada a la instancia de validación, no una medición de este consumo.

5.6.3. Archivos de seña

Una petición de pose es una lectura, no una generación. El contrato es el de la Sección 5.2.3. Las réplicas no guardan estado y leen la misma carpeta. El escalador de este grupo mira la cantidad de lecturas, no la cola del traductor. La ruta de salud queda aparte de la ruta del archivo. La caché del cliente sigue como primer recurso.

5.6.4. Monitoreo

Un recolector de métricas observa la latencia por oración, la memoria de cada contenedor del traductor, los errores de red y los errores de lectura del servicio de archivos. Esas series ajustan los dos escaladores.

5.6.5. Qué queda fuera de las pruebas

El prototipo no crea el balanceador, ni los escaladores, ni el volumen compartido. ILSA consulta una instancia del traductor, con los dos canales en un solo proceso, y una instancia del servicio de archivos. Esas dos publicaciones se describen en la Sección 6.6. La carga concurrente y el auto-escalado no entran en la validación.

Capítulo 6

Implementación y Desarrollo

6.1. Estructura del código

La arquitectura del sistema se organiza de forma modular para facilitar la escalabilidad, el mantenimiento y la separación de responsabilidades entre los componentes. El proyecto utiliza una arquitectura distribuida basada en microservicios y clientes livianos. La lógica se divide en dos bloques principales: un backend central encargado del procesamiento con inteligencia artificial, renderizado y comunicación en tiempo real, e interfaces de cliente compuestas por una extensión de navegador web y un empaquetador nativo.

6.1.1. Organización del Repositorio

El repositorio fuente contiene la totalidad de los módulos del sistema, recursos gráficos, archivos de configuración de entorno e instrucciones de despliegue. A continuación, se presenta la distribución lógica de directorios dentro de la rama principal del proyecto:

```
.
|-- assets/              # Gráficos de la aplicación
|-- extension/           # Extensión web para Chromium
|   |-- content/         # Scripts inyectados en Google Meet
|   |-- lib/             # Captura de video y MediaPipe
|   |-- vendor/          # MediaPipe Wasm de terceros
|   |-- background.js    # Service Worker de mensajería
|   |-- manifest.json    # Permisos y manifiesto V3
|   `-- offscreen.js     # Proceso fuera de la pestaña
|-- packaging/           # Compilación del ejecutable
|-- scripts/             # Evaluación y latencia
|-- src/                 # Backend en Python
|   |-- avatar/          # Animación del avatar
|   |-- backend/         # API FastAPI y Uvicorn
|   `-- classifier/      # Inferencia de señas
|-- LSABackend.bat       # Arranque en Windows
|-- requirements.txt     # Dependencias de Python
`-- run_backend.py       # Entrada del servidor
```

6.1.2. Descripción de Componentes y Responsabilidades

Cada directorio y paquete del sistema cumple una función específica dentro del flujo de inferencia y visualización:

- **Núcleo del Backend (`src/backend/`)**: Implementa la API REST y los canales WebSocket mediante FastAPI y Uvicorn. Administra las sesiones concurrentes para usuarios oyentes y señantes y coordina el flujo continuo de datos de landmarks y la devolución de traducciones o poses.
- **Módulo Clasificador de Señas (`src/classifier/`)**: Contiene la arquitectura de la red neuronal en PyTorch (`arch.py`) y los archivos de pesos entrenados (`tinyskeleton_best.pth`). Esos pesos corresponden a 100 clases, 16 fotogramas, 128 dimensiones, 4 cabezas de atención y 2 capas Transformer, entrenado con 100 clips por clase. Su función principal consiste en recibir secuencias temporales de puntos de referencia corporales y clasificar el gesto correspondiente en LSA.
- **Motor de Avatar esquelético (`src/avatar/`)**: Administra la lógica de animación y comunicación remota con servicios de representación gráfica. Traduce texto o tokens interpretados en secuencias de animación esquelética para su renderizado.
- **Extensión de Navegador (`extension/`)**: Actúa como la interfaz de usuario en tiempo real sobre Google Meet. Utiliza un *Service Worker* (`background.js`) para la gestión del estado, un entorno *offscreen* (`offscreen.js`) para procesar el flujo de video sin ralentizar la pestaña activa, y scripts de contenido (`content/meet.js`) para modificar la interfaz del usuario e integrar la ventana de interpretación.
- **Empaquetado y Distribución (`packaging/`)**: Contiene las configuraciones para PyInstaller (`LSABackend.spec`) y scripts automatizados en sintaxis Batch y Python. Permite generar la versión ejecutable del servidor sin requerir la instalación previa de un entorno Python en la máquina del usuario final.
- **Herramientas de Benchmarking (`scripts/`)**: Proporciona utilidades como `bench_latency.py` para medir la latencia punto a punto de la red neuronal y del canal de transmisión de datos.
- **Instrumentación de latencia:** El motor ILSA registra, por emisión, los cinco relojes de la Sección 3.1.3 (pausa, cola, inferencia, transporte y entrega) en CSV y en JSONL. Esa traza alimenta la verificación de RNF-01 en la Sección 7.3.1.

6.1.3. Convenciones de Nomenclatura y Estándares

El desarrollo adopta convenciones estándares según el lenguaje y entorno de ejecución:

- **Python**: Se aplican las directrices de estilo PEP 8. Los nombres de módulos, paquetes y funciones utilizan `snake_case`, mientras que las clases se definen en `PascalCase`. Las constantes globales se expresan en `UPPER_SNAKE_CASE`.
- **JavaScript/HTML/CSS**: Los scripts de la extensión siguen el estándar ECMAScript 6+ con sintaxis `camelCase` para variables y funciones. Los archivos de hoja de estilo e interfaces DOM emplean la convención `kebab-case`.

6.1.4. Archivos de Configuración y Entorno de Pruebas

El proyecto integra múltiples mecanismos de configuración para asegurar su reproducibilidad:

- **`manifest.json`**: Define los permisos del navegador, scripts de fondo y recursos accesibles desde la Extensión Web bajo la especificación Manifest V3.
- **`requirements.txt`** y **`requirements_exe.txt`**: Especifican las librerías necesarias con sus versiones fijadas tanto para el entorno de desarrollo como para la compilación final.
- **`mapeo_clases.json`**: Archivo de configuración alojado en `src/classifier/weights/` que vincula los índices numéricos de salida de la red neuronal con las etiquetas textuales del vocabulario LSA.

El entrenamiento del clasificador, el servicio semántico en la nube, el entrenamiento de cada canal y el servicio de poses del avatar viven en ramas propias. Su función y su disposición están en el Apéndice B.

6.2. Implementación del subsistema de reconocimiento gestual

6.2.1. Preprocesamiento de datos y pipeline de extracción

El pipeline procesa fotogramas de entrada mediante *MediaPipe Holistic* para estructurar un vector instantáneo de 225 coordenadas (33×3 de pose, 21×3 de mano izquierda y 21×3 de mano derecha). Para corregir discrepancias en la relación de aspecto de las cámaras ($4:3$ versus $16:9$), se aplica un reencuadre con bandas de relleno a una resolución uniforme de 853×480 píxeles previa extracción. Cada vector se centra en el punto medio de los hombros y se normaliza dividiendo por la distancia inter-hombros.

6.2.2. Arquitectura neuronal TinySkeletonClassifier

La clasificación de la secuencia temporal de 16 fotogramas se ejecuta mediante el modelo *TinySkeletonClassifier*, cuya implementación comprende:

1. Dos capas de convolución unidimensional temporal (núcleo de tamaño 3, 128 canales, normalización por lotes, activación ReLU y regularización Dropout de 0,2).
2. Un módulo de codificación posicional sinusoidal sumado a la representación de 128 dimensiones.
3. Un codificador Transformer de 2 capas con 4 cabezas de atención.
4. Una capa de agregación por atención (*Attention Pooling*) que asigna un escalar de relevancia a_t a cada fotograma t y reduce la secuencia temporal a un vector único de 128 dimensiones:

$$\mathbf{v} = \sum_{t=1}^{T} a_t \mathbf{h}_t \tag{6.1}$$

5. Una capa densa lineal final hacia las 100 clases del vocabulario gestual.

La Figura 6.1 resume esa sucesión y la forma del tensor en cada etapa.



Figura 6.1: Forma de los tensores en *TinySkeletonClassifier*. La entrada reúne 16 fotogramas de 225 coordenadas de pose y manos. Dos convoluciones de núcleo 3, con normalización por lotes y ReLU, la llevan a 128 canales. La codificación posicional y el Transformer de dos capas conservan 16×128. El *attention pooling* resume la secuencia en un vector de 128 dimensiones y la capa final distingue 100 señas.

Los pesos que carga ILSA son los de `model_new_2026_10_20`. El entrenamiento usó 8 000 secuencias de entrenamiento y 2 000 de validación. La exactitud de validación fuera de línea de ese checkpoint es del 99,0 %. La evaluación en cámara de ese mismo archivo se informa en la Sección 7.2.1.

6.3. Implementación del subsistema de traducción semántico bidireccional

6.3.1. Selección y especificación de la suite de modelos de lenguaje

Para la traducción semántica bidireccional se adaptaron modelos de lenguaje pequeños de entre 0,5B y 3B parámetros:

- **Familia Qwen2.5:** Variantes de instrucciones de 0,5B, 1,5B y 3B parámetros.
- **Familia Llama 3.2:** Variante de instrucciones de 1B parámetros.
- **Familia SmolLM2:** Variante de instrucciones de 1,7B parámetros.

En el servidor, cada canal arranca con Qwen2.5 de 1,5B y, si ese archivo no está disponible, con Llama 3.2 de 1B. El contrato de esas rutas está en la Sección 5.2.2.

6.3.2. Régimen de entrenamiento y ajuste LoRA

El canal directo y el canal inverso comparten el mismo procedimiento de entrenamiento. El corpus se divide en un 85 % para entrenamiento y un 15 % para evaluación. Esta separación utiliza una semilla fija para asegurar que cualquier prueba sea reproducible en las mismas condiciones. Cada ejemplo del conjunto de datos representa una interacción de tres partes: las instrucciones iniciales del sistema (*prompt*), el mensaje del usuario y la respuesta esperada. La ventana de procesamiento admite hasta 2048 tokens.

Para adaptar el modelo a la lengua de señas sin modificar sus miles de millones de parámetros originales, se utiliza la técnica LoRA. Se eligió un rango de adaptación $r = 16$ y un factor de escala $\alpha = 32$. La relación $\alpha/r = 2$. Estos valores responden a la configuración recomendada para modelos de este volumen.

Las matrices que reciben entrenamiento son las cuatro capas de proyección de atención y las tres capas de la red de avance. En cambio, la tabla de tokens de entrada y la capa de salida permanecen fijas. Esta elección se debe a que el vocabulario base del modelo no cambia; la tarea solo necesita reorganizar y conectar palabras que la red ya conoce.

El tamaño de lote efectivo se fijó en 16 ejemplos por cada paso del optimizador. Dado que el modelo de 3B parámetros consume más memoria, su micro-lote se redujo a 2 ejemplos y los gradientes se acumulan durante 8 pasos. Los modelos más pequeños utilizan micro-lotes de 4 ejemplos y acumulan durante 4 pasos. De esta forma, el paso matemático del optimizador es idéntico entre todas las opciones y los resultados son comparables.

El *learning rate* es de 3×10^{-4} para el modelo de 3B y de 5×10^{-4} para las versiones más pequeñas. Se asignó un valor menor al modelo grande porque un ajuste brusco en una red con más capas provoca inestabilidad en el aprendizaje. El optimizador elegido es AdamW cuantizado a 8 bits. Esta elección reduce el uso de memoria RAM en la GPU, ya que el estado del optimizador se guarda en 8 bits y convive sin problemas junto al modelo base de 4 bits. El decaimiento de pesos de 0,01 evita que los adaptadores crezcan demasiado y memoricen los datos. Se incluyen 10 pasos de calentamiento para que la tasa de aprendizaje suba de forma gradual desde cero, lo que evita desajustes iniciales bruscos en las matrices de LoRA que arrancan con valores al azar. Luego, un planificador de tipo coseno reduce de forma suave la tasa de aprendizaje hacia el cierre del entrenamiento para afinar los ajustes finales.

Para acelerar el entrenamiento y no perder capacidad de cómputo, la función de pérdida no evalúa todo el diálogo. Una máscara excluye del cálculo directo de la pérdida las posiciones del *prompt* y de la entrada, y deja el error sobre la respuesta deseada. La respuesta sigue condicionada por esos tokens durante el entrenamiento. De este modo, la red concentra su aprendizaje en la traducción.

Por último, antes de ingresar al entrenamiento, los datos reciben una limpieza específica. Las letras sueltas y consecutivas del dataset se unen en un solo token con mayúscula inicial, y los dígitos sueltos forman un número consolidado. Tres letras idénticas seguidas se reducen a dos para corregir errores de tipeo o captura. Las secuencias de palabras sin separación previa, como *Mar del Plata*, se agrupan en un único token *Mardelplata* para que la falta de espacios no confunda al modelo durante el aprendizaje.

Procedimiento de evaluación y post-entrenamiento

Al cierre del entrenamiento, el 15 % del conjunto de datos reservado se evalúa sobre el binario en formato GGUF cuantizado a 4 bits. Esta prueba mide el rendimiento real sobre el servidor de inferencia en producción, y no el del modelo en precisión flotante original.

La evaluación configura los siguientes parámetros generales de generación y medición:

- **Determinismo:** Una temperatura de valor 0 anula la aleatoriedad en la generación de texto y fuerza al modelo a elegir siempre el token de mayor probabilidad matemática. De esta forma, las respuestas resultan estrictamente reproducibles.
- **Control de repetición y parada:** Se aplica una penalización de repetición de 1,05 para desincentivar que el modelo entre en bucles de palabras o glosas idénticas durante la inferencia. El proceso se detiene ante la aparición de un salto de línea o de la marca de fin de turno, lo que evita el consumo innecesario de cómputo.
- **Múltiples referencias:** Cuando existen varias salidas válidas para una misma entrada, el sistema las agrupa a todas como referencias aceptadas para no penalizar variantes correctas.
- **Métricas de rendimiento:** La prueba registra el acierto exacto, el acierto tras normalizar puntuación y mayúsculas, y cinco métricas de texto: BLEU, ROUGE-L, METEOR, `BERT Score` y `Comet Score`. BLEU cuenta n-gramas compartidos con la referencia. ROUGE-L cuenta la subsecuencia común más larga. METEOR alinea palabras, raíces y sinónimos. `BERT Score` mide la cercanía de los vectores de los tokens. `Comet Score` estima la calidad de la traducción a partir de la entrada, la salida y la referencia. Van juntas porque el solapamiento de palabras no alcanza para decidir si se conservó el sentido. El detalle de cada una está en la Sección 3.3. La latencia se registra aparte, como tiempo de respuesta.

6.3.3. Canal directo (LSA → Español)

El par de este canal es glosas hacia una oración en español rioplatense. La entrada del usuario antepone «Glosas:» a la secuencia ya compacta. La consigna del ajuste pide una sola oración, sin notas ni datos ajenos a la entrada:

- El español es rioplatense y usa voseo. La oración incorpora artículos, preposiciones y concordancia verbal. La traducción no es palabra por palabra.
- Los pronombres de entrada son `YO`, `VOS`, `EL`, `NOSOTROS` y `ELLOS`. Sin una marca de género al lado, el sujeto queda tácito.
- La letra `A` inmediata al sustantivo o al pronombre marca femenino, y la letra `O` marca masculino. Un parentesco sin esa letra se considera masculino.
- `MIO` y `TUYO` expresan posesivos del español.
- `PASADO` o `AYER` exigen pretérito. `FUTURO` exige tiempo futuro. Sin marca temporal, el verbo queda en presente.
- `DONDE`, `QUIEN`, `COMO`, `QUE`, `CUANDO` y `CUANTOS` producen una pregunta directa, con signos de apertura y de cierre.
- `NO` junto al concepto o al verbo produce una negación.
- Una palabra de la secuencia en capitalización de nombre, o una cifra, se copia tal cual en la oración. La consigna prohíbe sustituirla o alterarla.

- La entrada sigue el orden *Topic-Comment* (tiempo, lugar, sujeto, sustantivo, posesivo, numeral, verbo y pregunta). La oración de salida reordena esa secuencia según la sintaxis del español.

Para la evaluación post-entrenamiento de este canal, se establece un límite máximo de 24 tokens del modelo para la salida (Sección 3.1.6). Esta restricción responde a que las oraciones traducidas del vocabulario base son breves, y frena la generación de texto infinito en caso de un fallo en la red.

6.3.4. Canal inverso (Español → LSA)

El par invierte al canal directo: la entrada es la oración y la respuesta son las glosas. La entrada antepone «Español:». La máscara deja el error sobre las glosas. Pide una sola secuencia, separada por espacios, sin notas, sin el prefijo «Glosas:» y sin artículos ni preposiciones del español:

- Las glosas léxicas van en mayúsculas sostenidas. Un nombre, un apellido o una calle va en capitalización de nombre, como `Patricia` o `Moreno`, y es una sola palabra de la secuencia, nunca letra por letra. Una cifra, como un documento o una cantidad, también es una sola palabra, no un dígito por espacio.
- El orden es tiempo, lugar, sujeto, sustantivo, posesivo, numeral, verbo y, al cierre, la partícula interrogativa o la negación.
- Los pronombres se explicitan aunque el español los omita. Tengo un cuchillo.” → `YO CUCHILLO TENER`. El mapeo es yo → `YO`, vos o tú → `VOS`, él → `EL O`, ella → `EL A`, nosotros → `NOSOTROS`, nosotras → `NOSOTROS A`, ellos → `ELLOS O` y ellas → `ELLOS A`.
- El género del parentesco usa el sustantivo en masculino más `O` o `A`: hermana → `HERMANO A`, hija → `HIJO A`, esposa → `ESPOSO A`. Un sustantivo de persona sin marca de género queda en masculino y suma `O` si el español lo explicita.
- `MIO` y `TUYO` cubren los posesivos. Me llamo X” o Mi nombre es X” → `YO NOMBRE` más el nombre.
- El pretérito o el copretérito colocan `PASADO` al inicio. El futuro, o la perífrasis voy a”, coloca `FUTURO`. El presente sin marca no agrega una glosa temporal.
- Las preguntas de dónde, quién, cómo, qué, cuándo y cuántos dejan `DONDE`, `QUIEN`, `COMO`, `QUE`, `CUANDO` o `CUANTOS` al final. La negación deja `NO` al final, junto al verbo.

Mediante estas reglas, el modelo aprende a emitir un nombre propio o una cifra como una sola palabra de la secuencia, y no como una cadena de letras. En la evaluación, el límite de generación de este canal es de 48 tokens del modelo, frente a 24 en el canal directo (Sección 3.1.6). Ese número cuenta piezas del tokenizador. Un nombre o una cifra larga es una palabra de la secuencia y, aun así, el tokenizador puede partirla en varias de esas piezas. El deletreo letra por letra ocurre después: si el avatar no encuentra el archivo de esa palabra, pide un clip por letra, con otro tope, de 24 clips.

6.3.5. Cuantización GGUF Q4_K_M y estimación de memoria

Los modelos ajustados se fusionaron y convirtieron a binarios GGUF con cuantización `Q4_K_M`. La memoria RAM requerida durante la inferencia se calcula mediante la relación:

$$\text{RAM} \approx \mathbf{W} + KV_{\text{fp16}}(ctx = 2048) + \text{Buffer} \tag{6.2}$$

donde $\mathbf{W}$ representa el tamaño de los tensores cuantizados y KV_{fp16} la caché de claves y valores para 2048 tokens de contexto. El buffer de activaciones es la memoria temporal de un paso de generación. Ahí entran el estado oculto de cada capa, las puntuaciones de atención de ese paso y el resultado intermedio de la red de avance. El paso siguiente reutiliza ese mismo espacio. La caché KV conserva, para el contexto, las claves y los valores ya calculados. La ecuación deja $\mathbf{W}$ en 4 bits por parámetro, esto es $\sim 0{,}5$ bytes, y cuenta solo el entero de cada peso. En la práctica el archivo GGUF pesa más: cada bloque guarda también la escala y el mínimo, `v_proj` y `down_proj` quedan en `Q6_K`, las normalizaciones siguen en 32 bits, y el archivo incluye la cabecera y el vocabulario. La tabla de `llama.cpp` no separa esas partes. Publica un solo promedio, el tamaño del GGUF dividido por la cantidad de parámetros, medido sobre Llama 3.1 8B: para `Q4_K_M` da 4,8944 bits por peso y un archivo de 4,58 GiB. En un modelo de ese tamaño la cabecera y el vocabulario casi no mueven el promedio. En los cinco modelos de este trabajo el cociente es otro: los archivos y la memoria que resulta de sumarles la caché están en las Tablas 7.13 y 7.14.

6.4. Integración en Google Meet y arquitectura de la extensión

6.4.1. Secuencia completa de inferencia y comunicación multidireccional

El motor local de la plataforma determina el modo operativo antes de cualquier interacción con la extensión de navegador. A través del panel de control, el usuario configura la lateralidad (diestro o zurdo) y el canal de comunicación (Sordo u Oyente). Las acciones de encendido, reinicio o apagado retornan a la pantalla de carga. La interfaz principal solo habilita sus funciones si el traductor semántico y el servidor de posturas responden correctamente.

El proceso principal escucha peticiones en una dirección de red local. Dado que las políticas de seguridad de Google Chrome identifican esa dirección como una red interna restringida, la página de la llamada no puede comunicarse con ella de forma directa. Por este motivo, el módulo en segundo plano de la extensión opera como intermediario, mientras que el documento secundario sin pantalla y la ventana emergente —al pertenecer al mismo origen de la extensión— entran en contacto directo con el motor local.

Durante la navegación en una llamada activa, el módulo en segundo plano efectúa una consulta inicial sobre el estado del sistema. Si el motor local se encuentra encendido, la extensión inyecta herramientas en dos entornos aislados del navegador:

- **Entorno de soporte:** Gestiona la interfaz flotante, las preferencias del usuario y el envío de solicitudes de red.
- **Entorno de la página principal:** Encargado de la renderización en pantalla y de tomar el control de la transmisión de la cámara web.

El sistema reitera la validación de estado cada dos segundos. Si el traductor no se encuentra listo, la interfaz solicita el reinicio del servicio y detiene el flujo. La modalidad seleccionada determina el canal activo, con la restricción de que el modo Sordo y el modo Oyente nunca se ejecutan en simultáneo.

El ciclo continuo de inferencia, procesamiento y comunicación se articula a través de las siguientes etapas:

1. **Gestión de la señal de captura en cámara única:** La extensión intercepta y sustituye la función nativa de acceso a la cámara. El dispositivo físico realiza una única apertura y deriva su señal de video hacia un elemento oculto dentro de la página. La llamada de la plataforma no recibe este flujo directo; en su lugar, recibe la señal sintética de un lienzo virtual a 24 cuadros por segundo. El transmisor de video de la llamada sustituye su fuente original por la de este lienzo. Al apagar la extensión, la página devuelve la señal nativa a la llamada sin detener abruptamente el hardware. Esta estrategia evita errores de dispositivo ocupado o no disponible, los cuales ocurren cuando la llamada exige el control exclusivo de la cámara física.

2. **Transmisión de fotogramas hacia el clasificador (Modo Sordo):** En la modalidad para personas sordas, la extensión extrae las imágenes de la cámara oculta y las comprime en un formato liviano con resolución y calidad reducidas. El sistema envía fotogramas y aguarda la confirmación antes de despachar el siguiente. El mensaje viaja internamente a través del módulo en segundo plano sin abandonar la computadora ni ingresar de forma directa al motor principal ILSA. Si la cámara de la llamada se cierra, la página emite un aviso para reiniciar el proceso de captura y recorte.

3. **Estimación de puntos articulares en el entorno aislado:** Las restricciones de seguridad del navegador impiden ejecutar la herramienta de visión artificial directamente en el módulo en segundo plano. Por esta razón, el sistema decodifica la imagen comprimida sobre un lienzo, extrae los píxeles y los transfiere hacia un documento aislado y protegido. Con esta información, el sistema estructura un vector cinemático. Si el usuario seleccionó la opción de lateralidad zurda, el servidor activa la inversión automática de ejes y el cruce de manos en el vector, ajustando los datos al modelo original que fue entrenado con muestras de mano derecha. La imagen comprimida no viaja en este envío.

4. **Criterio de corte y segmentación gestual:** El algoritmo de recorte opera dentro de la extensión en modo automático, sin evaluar el movimiento de los píxeles para evitar interferencias si hay personas caminando en el fondo. El inicio de la seña se registra al detectar una mano utilizable en la imagen. El cierre ocurre cuando las manos abandonan el encuadre, con un margen de tolerancia frente a eventuales pérdidas temporales de detección. El sistema no interrumpe una seña estática por falta de movimiento. El espacio de memoria guarda hasta 60 cuadros y realiza un muestreo de 16 posiciones antes de la transmisión.

5. **Clasificación local y filtrado mediante compuertas:** Cada seña segmentada se envía mediante un paquete de datos que contiene únicamente el vector de coordenadas. El clasificador procesa la información dentro del motor local. Si el nivel de confianza es inferior al umbral mínimo del 75 %, la predicción se descarta y la interfaz la presenta en color coral con la etiqueta «Poca confianza». Si alcanza el umbral pero la

compuerta de validación frena la entrada, la interfaz la despliega en tono gris como «No se agregó». Si la seña aprueba las validaciones, ingresa a la frase, el panel la destaca en cian con la indicación «En el mensaje» y el listado de la frase se actualiza.

6. **Finalización de enunciado y traducción semántica:** Tras un intervalo de 4 segundos sin actividad manual en la cámara, el sistema transmite la orden de cierre de frase. ILSA arma el español en el equipo si el enunciado es una sola glosa léxica, solo dígitos, o solo letras de dos en adelante y ninguna es I ni T. Una letra sola, un deletreo con I o T, OJO, VIERNES o AÑOS solas, y el resto de las frases van al servidor semántico. Si el módulo de traducción no emite respuesta, el motor conserva las señas aceptadas en formato de texto original sin reestructurar la oración.

7. **Emisión de salidas en el Canal Directo (Modo Sordo):** El sistema recibe la traducción en español y la publica en la página de la llamada. El módulo de video graba este texto sobre una franja oscura en la zona inferior del lienzo virtual, el cual conforma la imagen que los demás participantes observan. La capa visual del panel mantiene el texto visible durante 8 segundos. En la modalidad con salida por Subtítulo no se genera audio. En las configuraciones con Audio o Salida Doble, la página solicita una locución en español a la interfaz de voz nativa del navegador. Si una frase traducida es idéntica a la inmediata anterior, el sistema no la vuelve a reproducir.

8. **Procesamiento de entrada por voz en el Canal Inverso (Modo Oyente):** En la modalidad para personas oyentes el sistema captura el audio a través de un módulo de procesamiento sonoro. La extensión envía el bloque de audio comprimido al motor local, donde un modelo de reconocimiento de voz realiza la transcripción de forma totalmente privada y local. El modelo opera exclusivamente en idioma español. Si la conversión no produce texto, la interfaz solicita repetir la locución o recurrir al canal de texto.

9. **Procesamiento de entrada por texto en el Canal Inverso (Modo Oyente):** El sistema captura el contenido escrito sin pasar por el módulo de voz. La entrada de voz y la de texto pueden permanecer activas de forma concurrente; ambas representan distintas vías de ingreso para el mismo caso de uso y no constituyen un modo separado.

10. **Generación de glosas y renderización de animación sintética:** La cadena de texto en español ingresa al motor local. ILSA la parte por espacios. Si queda una sola palabra, la pasa a mayúsculas y esa cadena es la glosa: no consulta al traductor ni a un diccionario para tomar esa decisión. Si quedan dos o más palabras, la oración se envía al servicio en la nube y retorna estructurada como señas en el orden gramatical del sistema. El motor local solicita los archivos de animación cinemática correspondientes a cada seña, con un límite de hasta 24 elementos por enunciado. En ausencia del archivo para una seña específica, el sistema solicita los archivos de cada letra para realizar un deletreo dactilológico. Si el servidor de animaciones no responde, las señas se conservan en texto y el bloque visual permanece vacío. La extensión recibe los datos de movimiento ya unificados por el servidor. En la superficie del lienzo, la cámara del usuario ocupa la mitad izquierda y el modelo de esqueleto ocupa la mitad derecha. La llegada de una nueva animación no interrumpe la secuencia en ejecución, y la cola de reproducción admite hasta 20 bloques animados, descarta el

más antiguo al colmar su capacidad. Las señas procesadas también se graban sobre la franja inferior del mismo video para que el interlocutor las visualice directamente en la llamada.

6.4.2. Motor local ILSA (FastAPI)

El servidor local se implementó en Python mediante el framework asíncrono FastAPI y se ejecuta sobre un servidor Uvicorn. El servicio escucha en `127.0.0.1:8765`, únicamente en la interfaz de bucle local. Esa escucha impide las conexiones directas que llegan por las demás interfaces de red. Los procesos de la misma máquina pueden abrir el puerto, y el enlace no comprueba su identidad ni opera como autorización.

Para cumplir con las políticas de seguridad del navegador, el middleware CORS compara el encabezado `Origin` con tres valores literales y con una expresión regular. Los literales son `https://meet.google.com`, `http://127.0.0.1:8765` y `http://localhost:8765`. El script de contenido corre dentro de la página de Meet, de modo que sus peticiones presentan el primer origen. Los documentos propios de la extensión, cuyo origen comienza por `chrome-extension://`, se aceptan con la expresión regular `^chrome-extension://[a-p]{32}$`. Esa expresión describe el formato de un identificador de extensión de Chrome: 32 letras minúsculas de la `a` a la `p`. El navegador permite entonces leer la respuesta a cualquier extensión cuyo identificador tenga esa forma. La configuración no reserva el identificador de este prototipo.

La expresión regular se usa porque la extensión todavía no está publicada en la tienda del navegador y, por lo tanto, no tiene un identificador fijo. Al publicarla, la tienda asigna un identificador de ese mismo formato y lo conserva en las actualizaciones. En ese momento la expresión regular puede retirarse y el servidor puede admitir solo el origen literal `chrome-extension://` seguido de ese identificador.

Ese control no autentica al llamador. La API local no exige credencial, y un programa de la misma computadora puede consultar el puerto sin presentar el origen de una extensión. El servidor responde además con la cabecera de acceso a red privada (`Access-Control-Allow-Private-Network: true`), requerida cuando una página HTTPS realiza peticiones hacia el dominio local.

La emisión de voz sintética se desacopló mediante un hilo de ejecución secundario que consume una cola secuencial de mensajes, lo que evita bloqueos en la atención de las solicitudes HTTP de la API. El motor local se empaquetó como un ejecutable independiente de Windows (`ILSA.exe`) mediante PyInstaller, e incluye una interfaz gráfica desarrollada en Tkinter para permitir la selección de modo (Sordo u Oyente), la configuración de la mano dominante y el control de encendido del servicio.

6.5. Implementación del avatar

6.5.1. Vía descartada: retarget hacia un avatar VRM

Antes del diccionario de poses se probó un personaje VRM, con malla y huesos de longitud fija, animado por rotaciones. El fundamento del ensayo es el *retarget*: el paso de posiciones de *landmarks* a rotaciones de otro esqueleto. MediaPipe Holistic entrega posiciones, y el avatar exige rotaciones sobre su propio esqueleto. El visor se armó con Three.js. Para la mano se probó Kalidokit, una biblioteca de JavaScript para ese *retarget*. Toma los *landmarks* de MediaPipe y calcula rotaciones de Euler del esqueleto y pesos de

blendshape del rostro. En la mano resume cada dedo en un único ángulo de rizo. También se probó una cinemática inversa que lleva el extremo del hueso hasta el punto medido.

La prueba no se retuvo. El desarrollo no contaba con práctica suficiente de animación tridimensional, y las bibliotecas de *retarget* no sostuvieron de forma estable el paso de la pose al esqueleto VRM. El emparejamiento fallaba con frecuencia porque la pose pierde la profundidad en el eje Z: la coordenada de la mano queda relativa a la muñeca y en una escala distinta del desplazamiento en el plano, de modo que el hueso no recibe una distancia utilizable. Si se respeta la longitud del hueso, la yema no cae sobre el punto medido. Si la cinemática inversa alarga el miembro hasta ese punto, la mano pierde la configuración y, en el alfabeto manual, la letra cambia. Kalidokit, además, confunde configuraciones próximas.

El diccionario de `.pose` evita ese paso: dibuja los puntos medidos y no los convierte en rotaciones de otro cuerpo. Si más adelante se retoma el avatar tridimensional, una línea de mejora es filmar cada seña desde tres ángulos, para reconstruir la profundidad que una sola toma frontal no conserva.

6.5.2. Decisiones técnicas de la reproducción

La Sección 5.3 fija qué se muestra y por qué. Aquí quedan los valores con los que esas reglas se programaron. El servicio de poses entrega el archivo `.pose` pedido, sin recortarlo, sin llevarlo a una cadencia común y sin armar el puente. La preparación del clip y el empalme entre glosas corren en ILSA, una vez que los archivos ya llegaron. La extensión recibe la lista de fotogramas ya armada y la dibuja en la mitad derecha del lienzo. Elige, según el tiempo transcurrido, cuál de esos fotogramas le toca mostrar.

Valores del compositor. Antes del recorte, todos los clips se llevan a 30 fotogramas por segundo. Con esa tasa común, una espera de seis fotogramas dura lo mismo en cualquier seña. Si se dejara cada video a la cadencia con la que fue filmado, seis fotogramas de un clip a 24 por segundo durarían más que seis fotogramas de un clip a 30, y el empalme perdería la medida del tiempo.

El suavizado es un promedio exponencial. Si x_t es la posición recién medida y p_{t-1} es la pose ya aceptada en el fotograma anterior, la pose que se conserva es

$$p_t = \alpha\, x_t + (1 - \alpha)\, p_{t-1}, \tag{6.3}$$

con $\alpha = 0{,}50$. La medición nueva y la pose anterior pesan lo mismo. Ese peso es único para todo el diccionario.

El recuadro común mide 720×780 píxeles. El cuerpo se escala para que el ancho de hombros quede en 250 píxeles. Así, dos tomas de distinto encuadre ocupan el mismo espacio cuando se pegan una detrás de la otra.

El puente dura entre 4 y 8 fotogramas. Si las muñecas del final de una seña y del inicio de la siguiente ya están cerca, se usan 4. Si están lejos, se llega a 8, para que la mano no salte. Un punto cuya confianza queda por debajo de 0,15 sale en la lista como ausente, y el trazo del lienzo no lo une con el punto anterior.

La entrada, desde la postura de espera hasta el primer fotograma de la seña, y la salida, de vuelta a esa espera, duran cerca de seis fotogramas de la tasa de 30. A esa cadencia, cada uno de esos tramos ocupa cerca de un quinto de segundo. La secuencia se corta en el techo fijado en el diseño: 24 glosas y 1200 fotogramas.

Postura de espera. No hay un archivo aparte llamado reposo. El cuerpo y las manos de la espera se copian de los primeros fotogramas de una seña de referencia, el tramo en el que la persona todavía no empezó el gesto, y se escalan al mismo ancho de hombros del lienzo. El antebrazo queda casi horizontal, levemente hacia abajo, y la mano sigue esa línea.

Reflejo de la miniatura local. La videollamada muestra la miniatura de quien usa el sistema como un reflejo. La extensión deshace ese reflejo solo en ese video local. El esqueleto que viaja hacia la otra persona no se invierte. Las dos vistas quedan iguales: la cámara en la mitad izquierda y el esqueleto, sin darlo vuelta, en la mitad derecha.

6.6. Instancia de validación del prototipo mediante microservicios en la nube

Para la validación funcional del prototipo en un entorno de producción representativo, se completó el desacoplamiento del backend semántico y del servidor de poses del avatar hacia la infraestructura serverless de Google Cloud Run.

La arquitectura se dividió en dos microservicios independientes, empaquetados en contenedores Docker:

- **Servidor Semántico:** Este microservicio gestiona los modelos cuantizados a 4 bits (GGUF `Q4_K_M`) para la traducción directa e inversa. Se desplegó en la región *us-central1* con una asignación de 8 GiB de memoria RAM y 4 vCPU.
- **Servidor de Poses y Activos del Avatar:** Este microservicio entrega el diccionario de glosas como archivos `.pose`. La carpeta que viaja en la imagen reúne 551 archivos y ocupa cerca de 637 MiB. El proceso no los carga todos en memoria: responde el archivo que pide cada solicitud. Se desplegó en la región *southamerica-east1* con una asignación de 512 MiB de memoria RAM y 1 vCPU.

El proceso de despliegue e integración en la nube se llevó a cabo bajo los siguientes criterios técnicos:

1. **Empaquetamiento e integración continua (CI/CD):** Mediante la herramienta `gcloud builds`, el código fuente y las dependencias de cada rama se enviaron a Google Cloud Build para construir las imágenes de contenedor, las cuales quedaron almacenadas en Google Artifact Registry (`ilsa-repo`).
2. **Gestión dinámica de puertos y comprobación de estado:** A fin de evitar fallos de inicialización (*health-check timeouts*), cada contenedor se configuró para adaptar su punto de entrada a la variable de entorno `$PORT` inyectada de forma dinámica por la plataforma Cloud Run, que habilita la escucha en la interfaz `0.0.0.0` con un tiempo de espera de hasta 300 segundos para absorber la latencia de arranque inicial (*cold start*).
3. **Política de asignación de instancias y control presupuestario:** Con el propósito de optimizar el consumo de créditos y acotar los costos operativos dentro del marco del Trabajo Final de Grado, se configuró una política de instancias delimitada entre cero y una réplica (`-min-instances 0`, `-max-instances 1`). El escalado a cero libera los recursos cuando no existen llamadas activas.

El motor local ILSA y la extensión de navegador se enlazaron con las URLs públicas HTTPS de ambas instancias en Cloud Run. Esta integración confirmó la correcta transmisión de cargas textuales y la recuperación de activos para la sintaxis del avatar en tiempo pseudo-real y mantiene liberados los núcleos de cómputo y la memoria del dispositivo cliente.

Capítulo 7

Pruebas y validación

En el presente capítulo se expone la estrategia de verificación y validación del sistema de traducción en tiempo pseudo-real de LSA. Se detallan el plan de pruebas adoptado, la ejecución de los experimentos sobre los distintos componentes y el análisis cuantitativo y cualitativo de los resultados obtenidos frente a los requerimientos del proyecto. El objetivo central consiste en evidenciar la aplicación de criterios técnicos para evaluar el funcionamiento, el desempeño y la calidad del software, con una distinción clara entre planificación, ejecución y análisis de resultados.

7.1. Plan de pruebas

El plan de pruebas establece la estrategia sistemática para comprobar el correcto funcionamiento, el desempeño en tiempo real y la calidad del software desarrollado.

7.1.1. Alcance y tipos de pruebas

La evaluación del sistema abarca desde la recolección de datos y el entrenamiento de modelos de aprendizaje profundo hasta la integración completa de los componentes en una videollamada web. Se aplicaron los siguientes tipos de pruebas:

- **Pruebas unitarias y de componentes:** Verificación del extractor de puntos clave (*landmarks*), la lógica de filtrado gestual (`RepeatGate`), los clasificadores de señas y la cuantización de los modelos de lenguaje.
- **Pruebas de integración:** Evaluación de las interfaces entre la extensión de navegador (Manifest V3), el motor de inferencia local, las llamadas a API y el cliente de videoconferencia.
- **Pruebas de rendimiento y latencia:** Medición de tiempos de respuesta por etapa del canal de procesamiento, consumo de memoria en cliente y servidor, y tasa de fotogramas por segundo.
- **Pruebas de sistema de extremo a extremo (*End-to-End*):** Validación del circuito completo de traducción (LSA a español y español a LSA) en entornos de llamada reales.
- **Pruebas de robustez y casos límite:** Evaluación del clasificador frente a cambios de iluminación, distorsiones ópticas de cámara web, presencia de múltiples personas en el fondo y variabilidad en la velocidad de articulación gestual.

7.1.2. Entornos de evaluación

Las pruebas se ejecutaron en dos entornos representativos:

1. **Estación de desarrollo (GPU):** Utilizada para la evaluación estática fuera de línea, el entrenamiento de modelos y las pruebas de velocidad máxima de inferencia.
2. **Equipo objetivo de cliente (Laptop sin GPU):** Configuración de gama media estándar orientada a verificar el comportamiento en tiempo real, el consumo de CPU/RAM y la latencia perceptible por el usuario final.
3. **Infraestructura en la nube:** Instancia de Google Cloud Run (configuración de 4 vCPU y 8 GiB de memoria RAM) conectada mediante HTTPS para los servicios remotos de traducción semántica y reproducción de avatar.

7.1.3. Criterios de aceptación y trazabilidad

Cada caso de prueba se vinculó de manera directa con los requerimientos funcionales y no funcionales del sistema. En la Tabla 7.1 se sintetizan los criterios de aceptación principales definidos para la validación.

Cuadro 7.1: Matriz de trazabilidad entre requerimientos y criterios de aceptación para pruebas.

Módulo	**Requerimiento asociado**	**Criterio de aceptación**
Clasificador Gestual	Clasificación en tiempo real de vocabulario de 100 señas LSA.	Exactitud Top-1 en cámara real $> 75\,\%$ y Top-3 $> 90\,\%$. Latencia de inferencia $< 10\,\text{ms}$.
Traductor Semántico	Conversión bidireccional entre glosas y español fluido.	Exactitud sintáctica/semántica $> 85\,\%$, BLEU > 90, BERTScore > 98.
Motor Local	Filtrado de repeticiones y control de estado de gestos.	Supresión de falsos positivos y rebotes sin descartar repeticiones intencionales.
Integración Meet	Captura e inyección de video/texto sin alterar la videollamada.	Sin interrupción de la pista de video principal; renderizado de subtítulos sin pérdidas.

7.2. Resultados de la verificación

Esta sección presenta la evidencia empírica obtenida a partir de la ejecución del plan de pruebas sobre cada componente del sistema.

7.2.1. Evaluación del clasificador gestual (TinySkeletonClassifier)

El modelo de clasificación gestual se evaluó tanto en entornos estáticos fuera de línea como en pruebas en vivo frente a cámara web en la estación local.

Evolución del corpus y vocabulario

La incorporación de clases al modelo avanzó de manera paulatina conforme se consolidaron las muestras del dataset propio de LSA. En la Tabla 7.2 se detalla la progresión del vocabulario y las muestras recopiladas.

Cuadro 7.2: Progresión cronológica del conjunto de datos y vocabulario de LSA.

Nro	Clases	Muestras/Clase	Muestras Entrenamiento	Muestras Validación
1	91	50	3 640	910
2	94	60	4 512	1 128
3	97	60	4 656	1 164
4	100	60	4 800	1 200
5	100	100	8 000	2 000

Comparativa entre evaluación fuera de línea y cámara en vivo

En la Tabla 7.3 se expone la diferencia observada entre la exactitud de validación sobre el dataset estático y el rendimiento real obtenido frente a la cámara web.

Cuadro 7.3: Exactitud de validación fuera de línea frente al acierto en cámara. Las filas de 91 a 97 clases corresponden a una toma por seña. Las de 100 clases corresponden a diez tomas por seña.

Checkpoint	Dataset	Val. Top-1	Cámara Top-1	Cámara Top-3
16 fotogramas, 91 clases	1	97,36 %	61,5 %	83,5 %
32 fotogramas, 128/4/2	1	96,48 %	71,4 %	87,9 %
32 fotogramas, 64/2/1	1	97,36 %	67,0 %	90,1 %
Optuna v1, 32 fotogramas	1	97,69 %	66,0 %	84,6 %
Baseline 16f, luz ambiental, 94 clases	2	98,40 %	67,0 %	89,4 %
Baseline 16f, iluminación dirigida, 94 clases	2	98,40 %	86,2 %	95,7 %
Baseline 16f, 97 clases	3	98,28 %	77,3 %	94,8 %
100 clases, 10 tomas, luz adecuada	4	99,00 %	99,6 %	100,0 %
100 clases, 10 tomas, luz reducida	5	99,00 %	94,6 %	98,7 %

En las iteraciones de 94 clases, el control de la iluminación del entorno elevó el acierto en cámara del 67,0 % al 86,2 % en Top-1 sobre la misma arquitectura, sin modificar los pesos del modelo. Esta comparación no fija encuadre ni persona. La evaluación del checkpoint de 100 clases, descrita más adelante, sí conserva ambos factores y aísla el efecto de la luz.

Resolución temporal: 16 frente a 32 fotogramas

La Tabla 7.4 sintetiza los experimentos realizados para determinar la longitud óptima de la ventana temporal de entrada.

Cuadro 7.4: Evaluación de la resolución temporal en julio de 2026 (91 clases, 50 videos por clase).

Iteración	Arquitectura	Val. Top-1	Cámara Top-1	Cámara Top-3
A, 16 fotogramas	128 dim / 4 cabezas / 2 capas	97,36 %	61,5 %	83,5 %
B, 32 fotogramas	128 dim / 4 cabezas / 2 capas	96,48 %	71,4 %	87,9 %
C, 32 fotogramas	64 dim / 2 cabezas / 1 capa	97,36 %	67,0 %	90,1 %

A pesar de las ganancias iniciales observadas con 32 fotogramas, la incorporación de aumentaciones avanzadas de datos en agosto confirmó que la ventana de 16 fotogramas ofreció mayor capacidad de generalización sobre señas estáticas y redujo la redundancia temporal.

Impacto de la corrección de relación de aspecto

La estandarización del marco de captura desde proporciones 4 : 3 hacia 16 : 9 con bandas de relleno generó un impacto positivo directo en la clasificación, según se detalla en la Tabla 7.5.

Cuadro 7.5: Efecto de la corrección de relación de aspecto sobre 36 tomas de letras y dígitos.

Configuración de entrada	Exactitud Top-1
Captura original en relación 4:3	66,7 %
Corrección analítica en eje y ($\times$1,333)	88,9 %
Corrección simulada en ejes y y z	75,0 %

La transformación del eje y aportó un incremento de 22,2 puntos porcentuales en exactitud. Este ajuste corrigió confusiones sistemáticas, como el caso de la letra `I` clasificada previamente como `ojo`.

Evaluación en cámara del modelo de 100 clases

El prototipo integrado carga el modelo `model_new_2026_10_20`: 100 clases, 16 fotogramas, 128 dimensiones, 4 cabezas y 2 capas. El entrenamiento usó 100 clips por clase (8 000 secuencias de entrenamiento y 2 000 de validación) y alcanzó una exactitud de validación del 99,0 %, con pérdida de 0,028.

La prueba en cámara consistió en dos sesiones sobre ese mismo archivo. La mano, el encuadre y el extractor se mantuvieron fijos; solo varió la iluminación de la habitación. Quien ejecuta el script anota la etiqueta de luz adecuada o reducida, dado que el programa no mide la iluminancia. Cada sesión recorrió las 100 señas con diez tomas consecutivas (1 000 inferencias por condición), en modo automático y con mano derecha, sin omitir tomas.

La cámara se solicita a 1280×720 y entrega 640×480. El script rellena bandas hasta 853×480, el mismo marco 16:9 de los videos de entrenamiento. El extractor corresponde a MediaPipe Holistic en navegador (`@mediapipe/holistic@0.5.1675471629`): pose y dos manos, sin malla facial, vector de 225 coordenadas y resampleo a 16 fotogramas. Entre inferencias se estableció un segundo de espera.

La Tabla 7.6 resume los resultados. Las dos condiciones de iluminación cumplen el criterio de aceptación del plan (Top-1 mayor al 75 % y Top-3 mayor al 90 %).

Cuadro 7.6: Evaluación en cámara del modelo (`model_new_2026_10_20`). Diez tomas por seña, misma persona y mismo encuadre.

Métrica	**Luz adecuada**	**Luz reducida**
Top-1	99,6 % (996/1000)	94,6 % (946/1000)
Top-3	100,0 % (1000/1000)	98,7 % (987/1000)
Señas con 10 aciertos sobre 10	97	77
Errores que restan	4	54
Confianza media en aciertos	0,979	0,967
Confianza media en errores	0,632	0,771

Con luz adecuada, 97 señas registran diez aciertos en diez tomas. Restan cuatro errores y en todos ellos la seña solicitada ocupa el segundo lugar: `A` se confunde dos veces con `6`; `ahora_hoy` una vez con `tuyo`; `repetir` una vez con `esposo_a`. El subconjunto de 37 letras, eñe y dígitos alcanza un 99,5 % de Top-1.

Con luz reducida el Top-1 desciende 5,0 puntos y 20 señas dejan de completar diez aciertos sobre diez. El Top-3 permanece por encima del 98 %. Las caídas más marcadas en señas con diez o nueve aciertos con luz adecuada corresponden a `T` (3/10, hacia `I`), `como` (3/10, hacia `numero`), `años` (5/10, hacia `G`), `hola` (6/10, hacia `chau`) y `ahora_hoy` (6/10, con destinos diversos). En trece tomas la seña solicitada no entra al Top-3. En luz reducida la confianza media de los errores asciende a 0,771, indicador de que la red se equivoca con mayor certeza que con luz adecuada.

7.2.2. Resultados de los módulos de traducción semántica

La evaluación de los modelos de lenguaje reducidos (SLM) cuantizados en formato GGUF (`Q4_K_M`) se ejecutó sobre la partición final del corpus (119 oraciones para el canal directo LSA → Español y 102 oraciones para el canal inverso Español → LSA).

Evaluación del canal directo (LSA a Español)

En la Tabla 7.7 se presentan las métricas cualitativas y cuantitativas para el canal directo.

Cuadro 7.7: Canal directo (LSA → Español), resultados finales (119 oraciones de evaluación). Índices en escala 0–100.

Modelo	**Exacto**	**BLEU**	**ROUGE-L**	**METEOR**	**BERTScore**	**COMET**	**ms**
Qwen2.5 0,5B	92,44	95,58	95,78	95,51	99,16	97,47	141,05
Qwen2.5 1,5B	94,96	96,32	98,02	97,35	99,20	97,36	256,30
Qwen2.5 3B	95,80	97,88	98,39	97,84	99,20	97,56	514,38
Llama 3.2 1B	89,92	92,64	95,94	96,13	98,58	96,83	231,28
SmolLM2 1,7B	90,76	93,53	96,48	95,97	98,85	97,12	438,31

Evaluación del canal inverso (Español a LSA)

En la Tabla 7.8 se detallan las métricas correspondientes al módulo de estructuración en glosas para el canal inverso.

Cuadro 7.8: Canal inverso (Español → LSA), resultados finales (102 oraciones de evaluación). Índices en escala 0–100.

Modelo	Exacto	BLEU	ROUGE-L	METEOR	BERTScore	COMET	ms
Qwen2.5 0,5B	87,25	91,29	96,20	91,25	98,86	94,28	155,97
Qwen2.5 1,5B	91,18	94,14	97,17	92,75	99,22	95,98	290,82
Qwen2.5 3B	89,22	92,51	96,85	92,71	99,21	95,45	537,58
Llama 3.2 1B	89,22	92,66	96,36	92,89	99,13	95,29	245,70
SmolLM2 1,7B	77,45	82,90	91,54	86,31	97,68	92,37	458,39

El modelo Qwen2.5 1,5B alcanzó el equilibrio óptimo en ambas direcciones, con un acierto exacto del 94,96 % en canal directo y del 91,18 % en canal inverso, junto con tiempos de inferencia en CPU inferiores a 300 ms.

7.2.3. Pruebas de la lógica de control del motor local

La batería de pruebas diseñada para validar la lógica del filtro `RepeatGate` y los tiempos de corte de enunciado produjo los resultados consignados en la Tabla 7.9.

Cuadro 7.9: Resultados de las pruebas funcionales sobre la lógica de control del motor local.

Caso de prueba	Entrada ingresada	Resultado emitido
Dígitos repetidos	`1 1 1 2 2`	Aceptación total: `1 1 1 2 2`
Letra repetida tres veces	`A A A B`	Filtrado a máximo de dos: `A A B`
Rebote del clasificador	`HOLA HOLA`	Descarte de duplicado inmediato: `HOLA`
Repetición no consecutiva	`HOLA VER HOLA`	Conservación de la secuencia completa
Glosa repetida descartada a los 3 s	Glosa repetida	El cierre se desplaza a los 7 s
Movimiento continuo hasta 5 s	Sin glosa nueva	Cierre de enunciado a los 9 s
Deletreo dactilológico	`J U A N`	Conversión directa a «Juan.» sin consulta a nube
Deletreo con ambigüedad	`T I N A`	Derivación al traductor por coincidencia con `OJO`

7.2.4. Tratamiento de fallas de integración en Google Meet

Durante la evaluación sobre la interfaz de usuario en llamadas activas se identificaron inconvenientes técnicos que se solucionaron mediante intervenciones específicas de arquitectura, según se detalla en la Tabla 7.10.

Cuadro 7.10: Anomalías detectadas en la integración con Google Meet y soluciones aplicadas.

Falla detectada	**Intervención de ingeniería aplicada**
Bloqueo del inicio de video en Meet por intercepción permanente de cámara.	Activación del reemplazo de la pista únicamente con el motor ILSA encendido.
Pérdida de enlace al recargar la extensión con la pestaña de la reunión abierta.	Recarga automática de la pestaña de Meet ante el reinicio del contexto de la extensión.
Detección de falsas manos provocada por movimiento del torso.	Exigencia de un número mínimo de cuadros consecutivos con mano detectada antes de abrir el gesto.
Interferencia en la captura por movimiento de personas en el fondo.	Delimitación del área de actividad basada exclusivamente en los puntos articulares de las manos.
Sobrecarga de CPU en la transferencia de imágenes hacia la ventana *offscreen*.	Reducción de la resolución de los cuadros transferidos a 240 píxeles de ancho con compresión ajustada.

7.3. Métricas de calidad

La evaluación objetiva del sistema se consolidó mediante métricas cuantificables agrupadas en cuatro dimensiones principales: desempeño en tiempo real, eficiencia en el uso de recursos, calidad semántica y confiabilidad.

7.3.1. Pruebas de latencia y end-to-end en llamada real

Se ejecutó una sesión de pruebas end-to-end sobre la extensión integrada en videollamada web activa. Durante dicha sesión se capturaron eventos en ambos canales de traducción (directo e inverso), con registro de la latencia por etapa, el consumo de memoria del proceso cliente (RSS) y la tasa de fotogramas por segundo (FPS).

En la Tabla 7.11 se sintetizan los valores promedios, mínimos y máximos recolectados durante las ejecuciones de prueba.

Cuadro 7.11: Resumen de métricas de perfilado y latencia registradas durante la prueba del 29/10/2026.

Métrica de ejecución	**Canal Directo (LSA → Esp)**	**Canal Inverso (Esp → LSA)**	**Unidad**
Eventos registrados	9	7	oraciones
Pausa de cierre (configurada)	$4031{,}1 \pm 23{,}5$	0,0	ms
Tiempo de inferencia remota	1444,4 (896,7 – 2372,9)	1368,2 (826,6 – 2202,8)	ms
Tiempo de transporte de red	301,7 (273,4 – 336,4)	253,3 (254,2 – 328,2)	ms
Tiempo de entrega / renderizado	0,3 (0,2 – 0,6)	869,3 (105,9 – 1387,6)	ms
Latencia total de procesamiento	1712,9 (1233,3 – 2674,2)	2490,8 (105,9 – 3612,1)	ms
Consumo de memoria RAM (RSS)	168,8 (96,0 – 224,9)	231,2 (219,8 – 235,2)	MB
Tasa de procesamiento (Holistic)	5,8 – 12,1	N/A	FPS
Tasa de renderizado cliente	45,0 – 60,0	58,2 – 60,5	FPS
Cumplimiento RNF-01 ($< 2{,}0$ s)	77,8 % (7/9)	85,7 % (6/7)	aciertos

7.3.2. Evaluación del uso de recursos de hardware en el dispositivo cliente

A partir de los datos recopilados en cliente, se analizó el impacto del motor de inferencia local sobre los recursos de cómputo de la laptop objetivo sin GPU dedicada.

En la Tabla 7.12 se comparan los requerimientos de memoria y CPU observados durante la ejecución continua de la extensión.

Cuadro 7.12: Métricas de uso de RAM y uso de CPU en la estación cliente durante la prueba activa.

Estado del canal	**RAM Prom.**	**RAM Máx.**	**CPU Prom. (%)**	**CPU Máx. (%)**
Canal Directo	168,81	224,90	2,94 %	8,30 %
Canal Inverso	231,19	235,20	33,80 %	106,20 %

El análisis de perfilado muestra los siguientes comportamientos:

- **Eficiencia en memoria:** El canal directo presentó una huella de memoria reducida, la cual descendió de manera progresiva desde los 224,9 MB iniciales hasta un régimen permanente de 96,0 MB. El canal inverso mantuvo un consumo ligeramente superior pero acotado, con estabilización en ≈ 231 MB debido al almacenamiento en caché de las animaciones del avatar.
- **Carga del procesador:** Durante el procesamiento del canal directo el uso de CPU cliente permaneció reducido (2,94 % promedio), resultado que confirma la liviandad del clasificador gestual de 1,66 MB. En el canal inverso, el renderizado gráfico del avatar generó un incremento medio del uso de CPU hasta el 33,80 %, con picos transitorios del 106,20 % durante la decodificación inicial del audio entrante por el motor Vosk, con retorno inmediato al estado de reposo.

7.3.3. Consumo de memoria y huella del sistema en servidor

La cuantización en formato GGUF de 4 bits permitió reducir la huella de memoria del modelo de lenguaje a valores compatibles con despliegues eficientes. En la Tabla 7.13 y la Tabla 7.14 se sintetizan el tamaño de los archivos y el consumo estimado de memoria RAM en inferencia con un contexto de 2048 tokens.

Cuadro 7.13: Dimensiones de los archivos cuantizados y compresión alcanzada.

Modelo	**16 bits (GiB)**	**GGUF (MiB)**	**GGUF (GiB)**	**bits/parámetro**
Qwen2.5 0,5B	0,913	379,4	0,370	6,5
Qwen2.5 1,5B	2,869	940,4	0,918	5,1
Qwen2.5 3B	5,756	1 840,5	1,797	5,0
Llama 3.2 1B	2,291	770,3	0,752	5,3
SmolLM2 1,7B	3,185	1 006,7	0,983	4,9

Cuadro 7.14: Estimación del consumo de memoria RAM en inferencia (contexto de 2048 tokens).

Modelo	**Caché KV (GiB)**	**16 bits + caché (GiB)**	**GGUF + caché (GiB)**
Qwen2.5 0,5B	0,023	0,936	0,394
Qwen2.5 1,5B	0,055	2,923	0,973
Qwen2.5 3B	0,070	5,826	1,867
Llama 3.2 1B	0,062	2,354	0,815
SmolLM2 1,7B	0,375	3,560	1,358

En la réplica de producción desplegada en Cloud Run, ambos canales cargan Qwen2.5 1,5B. La columna GGUF más caché da 0,973 GiB para ese modelo, y los dos canales suman 1,946 GiB. Esa suma es el archivo medido más la caché KV en fp16, con contexto 2048. El buffer de activaciones de la Ecuación 6.2 queda fuera, y la cifra no es el RSS del proceso. Llama 3.2 1B queda en disco, a 0,815 GiB con su caché por canal, y entra en memoria solo si el modelo principal no carga. Los 8 GiB de la Sección 6.6 son la memoria asignada al contenedor, no esta suma.

7.3.4. Comparativa de rendimiento según arquitectura de hardware

Con el objeto de evaluar el comportamiento del motor local en un entorno con mayor capacidad computacional y contrastar estos resultados con los de la laptop objetivo (presentado en la Tabla 7.12), se ejecutaron las mismas pruebas sobre la estación de desarrollo de escritorio.

En la Tabla 7.15 se detallan la tasa de captura, el refresco de pantalla, el tiempo de inferencia local y el consumo de recursos de cómputo registrados en esta plataforma.

Cuadro 7.15: Métricas de procesamiento y uso de recursos en la estación de desarrollo de escritorio.

Métrica de ejecución	**Canal Directo**	**Canal Inverso**
Tasa de procesamiento (Holistic)	25,3 – 28,9 FPS	N/A
Tasa de renderizado de interfaz	194,0 – 200,6 FPS	194,0 – 200,6 FPS
CPU promedio (máximo)	0,87 % (7,20 %)	22,48 % (80,70 %)
Memoria RAM promedio (RSS)	544,42 MB	665,98 MB
Inferencia *TinySkeleton*	1,1 – 2,8 ms	N/A

El contraste entre los valores reportados en la Tabla 7.15 y los observados en el equipo portátil objetivo permite extraer las siguientes conclusiones técnicas:

- **Desacoplamiento de la tasa de captura gestual:** En el equipo de escritorio la extracción de puntos clave alcanza el techo de rendimiento del sensor ($\approx$ 27–28 FPS), en comparación con el rango de 5 a 12 FPS de la laptop. No obstante, la tasa de acierto del clasificador se mantiene estable en ambos entornos, hecho que valida la robustez del remuestreo temporal implementado en la lógica de control.
- **Impacto en el consumo de CPU y memoria:** El uso de procesador disminuye significativamente en la estación de desarrollo (0,87 % en directo y 22,48 % en inverso), lo que confirma que el costo computacional del motor local depende del hardware cliente. El mayor consumo de memoria RAM registrado en el equipo de escritorio ($\approx$ 544–666 MB) responde a las políticas de asignación de memoria y recolección de basura más permisivas del navegador Chromium sobre un sistema con mayor capacidad.
- **Fundamentación de la arquitectura híbrida:** La ejecución local de un modelo de lenguaje de 1,5B de parámetros requeriría más de 8 segundos de procesamiento en CPU aún en la estación de desarrollo. Por consiguiente, la delegación de la inferencia semántica a la infraestructura en la nube se consolida como la decisión de diseño idónea para garantizar respuestas en tiempo real con independencia de la capacidad de cómputo del usuario.

Capítulo 8

Evaluación

El presente capítulo expone la evaluación técnica, crítica e integradora del sistema desarrollado. Se analiza el nivel de cumplimiento de los objetivos planteados al inicio del proyecto sobre la base de las evidencias obtenidas en las etapas de experimentación, prueba y validación. Asimismo, se detallan las limitaciones metodológicas y tecnológicas observadas durante el desarrollo, y se examinan los riesgos técnicos y operativos inherentes a la solución con sus respectivas estrategias de mitigación.

8.1. Cumplimiento de objetivos

La valoración de los resultados se realiza a través de la trazabilidad directa entre los objetivos definidos en el Capítulo 2, los requerimientos especificados en el Capítulo 4 y las evidencias empíricas recolectadas en los ensayos de validación.

8.1.1. Evaluación del objetivo general

El objetivo general consistió en desarrollar un prototipo funcional de traducción bidireccional entre la LSA y el español rioplatense en tiempo pseudo-real, integrado en una plataforma de videoconferencia mediante una arquitectura híbrida cliente-nube.

El cumplimiento de este objetivo se alcanzó mediante la construcción exitosa de la extensión para navegadores Chromium conectada al motor local ILSA y a los servicios en la nube. El sistema procesa la interacción en dos sentidos: en el canal directo transforma la captura de cámara en subtítulos y voz sintética, mientras que en el canal inverso traduce voz o texto a la animación de un avatar esquelético. La arquitectura híbrida mantiene el procesamiento de visión en el cliente y la inferencia de lenguaje en la nube, con lo que se mitiga la asimetría comunicacional sin transferir la señal de video fuera del dispositivo.

8.1.2. Evaluación de los objetivos específicos

A continuación se analiza el grado de logro de cada uno de los objetivos específicos:

1. **Implementación del módulo local de visión y clasificación gestual:** *Estado: Alcanzado.* Se desarrolló el pipeline basado en MediaPipe Holistic para la extracción de 225 coordenadas articulares normalizadas y la red neuronal *TinySkeletonClassifier*. Las pruebas experimentales demostraron una exactitud del 99,0 % en la validación fuera de línea (sobre 2 000 secuencias de prueba) y un 99,6 % de acierto en pruebas

con cámara real bajo iluminación adecuada, lo que superó la meta del 75 % definida en el requerimiento RNF-03.

2. **Diseño e integración del servicio semántico para el canal directo (LSA → Español):** *Estado: Alcanzado.* Se ajustó y desplegó el modelo Qwen2.5 1,5B mediante la técnica LoRA y cuantización GGUF `Q4_K_M`. En las evaluaciones sobre oraciones fuera del conjunto de entrenamiento, el modelo logró un acierto exacto del 94,96 %, con métricas de fidelidad semántica elevadas (98,85 de BERTScore F1 y 97,12 de COMET score). La inferencia remota respondió en un tiempo medio de 1444,4 ms en la sesión integrada, en cumplimiento con el requerimiento RF-03.

3. **Diseño e integración del servicio semántico para el canal inverso (Español → LSA):** *Estado: Alcanzado.* Se estructuró el canal de retorno para reorganizar oraciones en español hacia el orden manual simplificado del prototipo (*Topic-Comment*). El modelo ajustado alcanzó un acierto exacto del 91,18 % en las pruebas del laboratorio semántico y una latencia media de inferencia remota de 1368,2 ms, lo que satisface el requerimiento RF-06.

4. **Implementación del módulo de visualización gestual mediante avatar:** *Estado: Alcanzado.* Se diseñó el motor de animación cinemática basado en archivos binarios `.pose`. El sistema reproduce hasta 24 clips por oración con suavizado exponencial y puentes de transición interpolados. Ante la ausencia de un archivo de seña, se activa automáticamente el subsistema de deletreo dactilológico. Se incorporó una cola de reproducción con capacidad para 20 animaciones, en cumplimiento con las especificaciones del requerimiento RF-07.

5. **Diseño de una arquitectura escalable en la nube:** *Estado: Parcialmente alcanzado.* Se diseñó la arquitectura elástica teórica con balanceo de carga, contenedores sin estado y escalado horizontal automático (Sección 5.6 y Apéndice C). Sin embargo, en concordancia con la restricción RC-02, la validación experimental no desplegó el clúster elástico multinodo, sino una instancia única por servicio en Google Cloud Run (`-max-instances 1`). El alcance efectivo alcanzado comprobó el desacoplamiento funcional de los microservicios, pero dejó la prueba de carga concurrente masiva como una línea futura.

6. **Resguardo de la privacidad de los datos biométricos:** *Estado: Alcanzado.* Se garantizó la ejecución local de la extracción de *landmarks* y de la clasificación de señas en el equipo cliente. La comunicación saliente hacia la nube se restringe a cadenas de texto (glosas u oraciones) o solicitudes de nombres de archivos, lo que evita de forma estricta la transmisión de flujo de video o coordenadas crudas fuera del dispositivo, en cumplimiento estricto del requerimiento RNF-02.

7. **Desarrollo de la extensión web e integración con Google Meet:** *Estado: Alcanzado.* Se construyó la extensión bajo el estándar Manifest V3 articulada con el motor local ILSA mediante la arquitectura desacoplada de entornos *offscreen* y *sandbox*. Se implementó la intercepción transparente del dispositivo mediante la inyección de un canvas sintético sobre la llamada de Google Meet (`captureStream()`), en cumplimiento con los requerimientos RF-08, RF-09 y RNF-05.

8. **Evaluación integral del sistema y medición de desempeños:** *Estado: Alcanzado.* Se ejecutó el protocolo de pruebas completo: medición de acierto por seña en

cámara real, evaluación cuantitativa de fidelidad semántica en ambos canales con cinco métricas estandarizadas, caracterización de tiempos de respuesta desagregados por etapas (pausa, transporte, cola, inferencia y entrega) y verificación del consumo de recursos en hardware cliente estándar.

La Tabla 8.1 sintetiza la trazabilidad entre los objetivos, los requerimientos asociados y los resultados obtenidos.

Cuadro 8.1: Matriz de trazabilidad entre objetivos, requerimientos y resultados.

Objetivo específico	**Requerimientos**	**Evidencia / Resultado obtenido**	**Estado**
1. Clasificador gestual	RF-01, RF-02, RNF-03	Exactitud fuera de línea de 99,0 %; acierto en cámara de 99,6 %.	Alcanzado
2. Canal directo	RF-03	Exactitud exacta de 94,96 %; BERTScore F1 de 98,85. Latencia media de 1444,4 ms.	Alcanzado
3. Canal inverso	RF-06	Exactitud exacta de 91,18 %. Latencia media de 1368,2 ms.	Alcanzado
4. Avatar e integración	RF-07	Renderización cinemática a 30 FPS, suavizado EMA y contingencia por dactilología.	Alcanzado
5. Arquitectura nube	RNF-04, RC-02	Diseño elástico documentado; validación en Cloud Run con réplica única.	Parcial
6. Privacidad biométrica	RNF-02	Extracción local; solo se transmiten cadenas de texto en los payloads HTTP.	Alcanzado
7. Extensión web	RF-08, RF-09, RNF-05	Inyección en Google Meet mediante Canvas sintético bajo Manifest V3.	Alcanzado
8. Evaluación integral	RNF-01, RC-05	Trazabilidad de latencia por etapas y perfilado de hardware cliente.	Alcanzado

8.2. Limitaciones

El desarrollo y el desempeño del prototipo se encuentran condicionados por delimitaciones técnicas, metodológicas y tecnológicas acordadas para acotar el alcance del proyecto de grado. La explicitación de estos factores establece los bordes reales del sistema:

- **Recorte léxico y ausencia de gramática no manual:** El vocabulario soportado por el clasificador se acota estrictamente a 100 señas aisladas (Apéndice A). Asimismo, el pipeline excluye la gesticulación facial, los movimientos de cejas y la postura del torso como rasgos gramaticales. Por lo tanto, el sistema no recupera la modulación afectiva o interrogativa que reside exclusivamente en los canales no manuales de la LSA.

- **Requisito de separación de gestos en encuadre:** El algoritmo de detección de pausas exige retirar la mano del encuadre de la cámara entre la ejecución de cada seña. Esta condición operativa resulta indispensable para segmentar correctamente los gestos aislados, aunque impone una mecánica de señado discontinua que difiere de la fluidez natural de la LSA.

- **Condiciones de captura y dependencia de MediaPipe:** La calidad del reconocimiento gestual depende de la correcta extracción de *landmarks* por parte de MediaPipe Holistic. Situaciones de oclusión severa entre las manos, desenfoque por movimiento rápido, iluminación deficiente o encuadres fuera del plano del torso incrementan la tasa de falsos positivos o la degradación de las coordenadas articulares.
- **Delimitación estricta de límites temporales:** El prototipo exige una pausa fija de inactividad manual de 4 segundos para determinar la finalización de una frase en el canal directo. Esta restricción introduce un tiempo de espera estructural previo al envío de la solicitud hacia la nube, lo que impide un flujo conversacional verdaderamente fluido o ininterrumpido.
- **Representación cinemática simplificada del avatar:** El ensamblado del avatar opera mediante la interpolación geométrica entre núcleos de gestos previamente grabados. Esta aproximación atenúa la discontinuidad espacial entre señas, pero no replica los fenómenos biomecánicos complejos de la coarticulación gestual presentes en el movimiento humano natural.
- **Dependencia de conectividad de red y penalización de arranque en el prototipo:** A pesar de ejecutar la clasificación gestual de forma local, el servicio semántico y la descarga de animaciones dependen de la conectividad hacia la nube. En el escenario de pruebas sobre infraestructura *serverless* (Cloud Run) configurada con escalado a cero (`min-instances 0`) para optimización de costos, la inicialización del sistema enfrenta una demora por el arranque en frío (*cold start*) de las instancias. Dado que el motor local exige la disponibilidad previa de los servidores para iniciar su operación, este retraso afecta exclusivamente al proceso de arranque del prototipo. En un entorno de producción, esta penalización inicial se elimina mediante el mantenimiento de instancias reservadas continuas.
- **Entorno cliente y compatibilidad de plataforma:** El empaquetado del motor local ILSA se optimizó específicamente para el sistema operativo Windows y navegadores Chromium. La ejecución concurrente de la videollamada, la extracción de puntos mediante WebAssembly y la clasificación en CPU exige el cumplimiento estricto de los requisitos mínimos de hardware (4 núcleos con instrucciones AVX2 y 8 GB de RAM), lo que limita su despliegue en dispositivos de menor capacidad computacional.

8.3. Riesgos detectados y estrategias de mitigación

El análisis del prototipo en un escenario proyectado hacia la producción evidencia la presencia de riesgos técnicos, operativos y evolutivos. En la Tabla 8.2 se sistematizan los principales factores identificados junto con sus correspondientes líneas de mitigación y propuestas de mejora ingenieril.

Cuadro 8.2: Análisis de riesgos técnicos, operativos y evolutivos con estrategias de mitigación.

Categoría	**Riesgo identificado**	**Descripción e impacto**	**Estrategia de mitigación / Línea de mejora**
Seguridad y Privacidad	Vulnerabilidad en la API local no autenticada	La escucha del servidor FastAPI en la red local sin tokens de autenticación expone endpoints a otros procesos del mismo sistema cliente.	Implementación de autenticación mediante *tokens* efímeros (JWT) intercambiados de forma segura durante el arranque de la extensión.
Evolución y Obsolescencia	Cambios en el DOM de Google Meet y APIs web	Actualizaciones en la plataforma de videollamada o restricciones adicionales en Manifest V3 pueden romper la inyección de scripts.	Abstracción de los selectores DOM en capas de desacoplamiento y mantenimiento periódico de la extensión en el repositorio.
Mantenibilidad	Dificultad para incorporar nuevas señas	El reentrenamiento del clasificador y la captura de nuevos archivos `.pose` requiere trabajo manual intensivo.	Construcción de herramientas de captura automatizadas y canalizaciones de integración continua (*MLOps*) para modelos de visión.

Capítulo 9

Conclusiones y trabajos futuros

El presente capítulo sintetiza los resultados técnicos del proyecto, evalúa el cumplimiento de los objetivos y analiza el impacto social de la solución. Asimismo, proyecta las líneas de continuidad e investigación futura derivadas de las limitaciones y aprendizajes observados durante el desarrollo del sistema.

9.1. Conclusiones

El desarrollo de este Trabajo Final de Grado permitió validar la factibilidad técnica y operativa de un sistema de traducción bidireccional entre la LSA y el español rioplatense en tiempo pseudo-real, concebido para operar sobre plataformas de videoconferencia de uso masivo como Google Meet. La arquitectura híbrida propuesta demostró que es posible desacoplar el procesamiento intensivo de visión en el equipo cliente de la inferencia de lenguaje en la nube, lo que garantiza el resguardo de la privacidad biométrica sin degradar la experiencia de comunicación.

9.1.1. Síntesis del cumplimiento de objetivos

La integración del sistema permitió alcanzar de manera satisfactoria el objetivo general y las metas específicas planteadas al inicio del proyecto. A partir del análisis detallado en el Capítulo 8, la evidencia empírica respalda el desempeño global del prototipo en sus componentes clave:

- **Canal directo (LSA → Español):** El pipeline de visión local sustentado en *TinySkeletonClassifier* demostró una capacidad de reconocimiento gestual sobresaliente (99,6 % de acierto Top-1 en cámara real bajo iluminación adecuada). Por su parte, la adaptación del modelo de lenguaje *Qwen2.5 1,5B* resolvió con fluidez la brecha sintáctica entre las glosas y el español rioplatense (94,96 % de acierto exacto) y mantuvo los tiempos de inferencia dentro del umbral de respuesta definido para el servicio.
- **Canal inverso (Español → LSA) y visualización:** El módulo semántico de retorno reestructuró adecuadamente la voz o texto del interlocutor oyente hacia el orden manual simplificado del prototipo (91,18 % de acierto exacto). La renderización cinemática mediante el avatar esquelético garantizó la representación gestual continua a 30 fotogramas por segundo, con una transición fluida entre clips y la activación efectiva del deletreo dactilológico frente a ausencias en el léxico grabado.

- **Arquitectura, privacidad e integración web:** Se concretó la ejecución transparente sobre Google Meet mediante una extensión Manifest V3 acoplada al motor local ILSA. La arquitectura aseguró la ejecución estrictamente local de la extracción de puntos clave, lo que impidió la transmisión de flujo de video o datos biométricos hacia la red. Si bien el despliegue en la nube se limitó a una instancia única por servicio en Google Cloud Run en cumplimiento del alcance presupuestario de las pruebas, se dejó completamente documentado el diseño elástico para producción.

9.1.2. Aportes técnicos e ingenieriles

El proyecto aporta decisiones de diseño y arquitecturas aplicadas que resuelven problemas concretos de la ingeniería de software y el aprendizaje profundo en dispositivos de borde:

- **Partición eficiente de cómputo híbrido:** La delegación de los modelos de lenguaje hacia la nube y la conservación del pipeline de visión en el cliente permitió ejecutar el sistema en hardware estándar sin placa gráfica dedicada y redujo el consumo de memoria RAM en el dispositivo cliente a un rango permanente de entre 96 y 231 MB.
- **Mecanismos de integración transparente en la Web:** La arquitectura basada en entornos *offscreen* y *sandbox* dentro de Manifest V3 resolvió las restricciones de ejecución de código dinámico en navegadores modernos. La inyección de un lienzo sintético (*Canvas captureStream*) permitió interceptar la llamada de Google Meet sin alterar la estabilidad del dispositivo de captura original ni requerir permisos especiales en la plataforma.
- **Estrategias de robustez en el procesamiento temporal:** La implementación de compuertas de filtrado de repetición (*RepeatGate*) y la estandarización del marco de captura mediante corrección de relación de aspecto eliminaron confusiones críticas en la clasificación de letras y números y aseguraron la estabilidad del flujo conversacional.

9.1.3. Impacto social y marco regulatorio

Este prototipo contribuye a reducir las barreras comunicacionales que afectan a la comunidad sorda en entornos digitales. En el marco de la Ley 27.710, que reconoce a la LSA como una lengua natural y legitima el derecho de las personas sorda a comunicarse en su idioma patrio, este desarrollo ofrece un antecedente tecnológico que fomenta la autonomía individual en interacciones de salud, trámites y educación virtual. La disponibilidad de un puente comunicacional bidireccional en tiempo pseudo-real constituye un avance tangible hacia la equidad de acceso en servicios donde la interpretación humana no se encuentra disponible.

9.2. Trabajos futuros

Las líneas de continuidad y expansión del sistema surgen de manera coherente a partir de las limitaciones identificadas en la etapa de evaluación y de la necesidad de escalar la solución hacia entornos de producción masiva. Se proponen los siguientes ejes de desarrollo:

9.2.1. Inclusión de la comunidad sorda y validación participativa

Incorporar a personas nativas de la comunidad sorda y a especialistas en LSA en el diseño, anotación del corpus y validación del sistema. La sustitución de la captura orientada por desarrolladores por un proceso de co-creación participativa garantizará una representación precisa de la variabilidad dialectal, la naturalidad de los movimientos y los matices culturales propios de la lengua.

9.2.2. Ampliación del dataset y captura multicámara

Regrabar y expandir el conjunto de datos gestual mediante tomas simultáneas desde múltiples ángulos y perspectivas espaciales. La integración de registros multicámara permitirá entrenar clasificadores con mayor invariancia frente a oclusiones mutuas de las manos, cambios en el punto de vista de la lente y variaciones en la postura del usuario respecto del plano frontal.

9.2.3. Incorporación de rasgos no manuales y reconocimiento continuo

Evolucionar desde la clasificación de señas aisladas hacia modelos de reconocimiento de lenguaje de señas continuo (CSLR). Esta etapa contempla la incorporación de la malla facial para capturar la gesticulación de boca, cejas e inclinación de torso, elementos indispensables para interpretar la modulación interrogativa, afectiva y temporal completa de la LSA.

9.2.4. Evolución hacia un avatar tridimensional (3D)

Sustituir la representación esquelética bidimensional actual por un avatar tridimensional con renderizado volumétrico y dinámicas de deformación de malla. La integración de motores gráficos en el cliente (tales como Three.js o Babylon.js) permitirá representar con exactitud física la profundidad, la rotación de muñecas y la orientación de las palmas, dimensiones fundamentales para una interpretación visual natural de la LSA.

9.2.5. Despliegue del clúster elástico multinodo en la nube

Implementar físicamente la infraestructura elástica proyectada en el diseño conceptual. Esto abarca la configuración de balanceadores de carga, la orquestación de contenedores sin estado con autosescalado horizontal según la longitud de cola de peticiones, y la verificación de la capacidad de respuesta frente a cargas de tráfico concurrente simultáneo.

9.2.6. Generalización a nuevas plataformas y ejecución local de SLM

Extender la compatibilidad de la extensión hacia otras herramientas de videoconferencia (como Zoom, Microsoft Teams o Webex) mediante capas de abstracción sobre el DOM. Asimismo, se plantea explorar la cuantización de modelos de lenguaje para ejecución local mediante WebGPU en el navegador, con el objetivo de eliminar la dependencia de servicios en la nube para el canal semántico y anular los tiempos de transporte de red.

9.2.7. Aportes de la inteligencia artificial generativa en el ciclo de desarrollo

En cumplimiento con las directivas institucionales relativas a la transparencia en el uso de tecnologías emergentes, se destaca el rol de las herramientas de Inteligencia Artificial IA generativa como asistentes en el proceso del proyecto. La incorporación de entornos como Cursor e interfaces de lenguajes como Gemini potenció la productividad técnica y aceleró las fases del desarrollo mediante los siguientes aportes concretos:

- **Optimización en la escritura y refactorización de código:** La asistencia en el entorno de desarrollo facilitó la navegación por el repositorio, el autocompletado sintáctico y la resolución eficiente de incompatibilidades en la API de Chrome Extensions (Manifest V3) y en los conectores asíncronos de FastAPI.
- **Generación y curación del corpus sintáctico:** La utilización de prompts parametrizados permitió construir borradores iniciales de pares de entrenamiento (español y glosas) bajo las reglas sintácticas del prototipo. Este proceso redujo los tiempos de preparación del dataset semántico, sujeto a la posterior revisión, corrección y validación manual por parte de los autores.
- **Consultoria técnica y exploración teórica:** Las herramientas actuaron como soporte de consulta para agilizar la asimilación de conceptos complejos, tales como los esquemas de cuantización `GGUF Q4_K_M` con `llama.cpp`, la configuración de adaptadores LoRA y la gestión de memoria en entornos *serverless*.
- **Revision de estilo y consistencia formal:** La asistencia generativa contribuyó a la revisión gramatical del informe, la estructuración de la documentación técnica y la corrección de estilo académico en los capítulos de la tesis.

Esta integración demostró que el uso responsable y supervisado de la inteligencia artificial constituye un catalizador valioso para la ingeniería de software, ya que complementa el criterio analítico, la toma de decisiones arquitectónicas y la responsabilidad intelectual de los autores.

Bibliografía

[1] República Argentina. *Ley 27.710 — Lengua de Señas Argentina*. Boletín Oficial. 2023.

[2] M. I. Massone y R. A. Martínez. *Curso de Lengua de Señas Argentina: Parte 1 y 2*. Cultura Sorda, 2012. URL: http://www.cultura-sorda.eu.

[3] S. K. Liddell y R. E. Johnson. «American Sign Language: The Phonological Base». En: *Sign Language Studies* 64 (1989), págs. 195-277.

[4] M. I. Massone. «El número y el género en la Lengua de Señas Argentina». En: *Signo & Seña* 2 (1993), págs. 73-99.

[5] C. Lugaresi et al. «MediaPipe: A Framework for Building Perception Pipelines». En: *Third Workshop on Computer Vision for AR/VR at IEEE CVPR*. 2019.

[6] Google. *MediaPipe: A Framework for Building Perception Pipelines*. https://google.github.io/mediapipe/. 2020.

[7] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang y M. Grundmann. *BlazePose: On-device Real-time Body Pose Tracking*. 2020. arXiv: 2006.10204 [cs.CV].

[8] F. Zhang et al. *MediaPipe Hands: On-device Real-time Hand Tracking*. 2020. arXiv: 2006.10214 [cs.CV].

[9] A. Moryossef, M. Müller y R. Fahrni. *pose-format: Library for viewing, augmenting, and handling .pose files*. https://github.com/sign-language-processing/pose. 2021.

[10] A. Vaswani et al. «Attention Is All You Need». En: *arXiv preprint arXiv:1706.03762* (2017).

[11] E. J. Hu et al. «LoRA: Low-Rank Adaptation of Large Language Models». En: *Proceedings of the International Conference on Learning Representations*. 2022.

[12] G. Gerganov. *llama.cpp: Port of Facebook's LLaMA model in C/C++*. 2023. URL: https://github.com/ggerganov/llama.cpp.

[13] W3C. *Media Capture and Streams*. https://www.w3.org/TR/mediacapture-streams/. 2024.

[14] W3C. *Media Capture from DOM Elements*. https://www.w3.org/TR/mediacapture-fromelement/. 2024.

[15] Google Chrome Developers. *Overview of Manifest V3*. https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3. 2024.

[16] Google Chrome Developers. *chrome.offscreen*. https://developer.chrome.com/docs/extensions/reference/api/offscreen. 2024.

[17] Google Chrome Developers. ***sandbox*** *(Manifest V3)*. https://developer.chrome.com/docs/extensions/reference/manifest/sandbox. 2024.

[18] Confederación Argentina de Sordos. *Señario de términos y expresiones en Lengua de Señas Argentina*. https://senario.cas.org.ar/. 2024.

[19] V. Barán. *LSApp: Uniendo dos mundos*. https://lsapp.com.ar/. 2018.

[20] Blessit. *Interpretación Lengua de Señas — Central de intérpretes*. https://www.blessit.net/interpretacion-lengua-de-senas. 2024.

[21] Ministerio de Justicia de la Nación. *Características de la comunicación por videollamadas — Línea 144*. https://www.argentina.gob.ar/caracteristicas-de-la-comunicacion-por-videollamadas. 2024.

[22] F. Ronchetti, F. Quiroga, C. Estrebou, L. Lanzarini y A. Rosete. «LSA64: An Argentinian Sign Language Dataset». En: *XXII Congreso Argentino de Ciencias de la Computación (CACIC)*. 2016, págs. 794-803.

[23] I. Mindlin. «Reconocimiento de Lengua de Señas con redes neuronales recurrentes». Tesis de grado. Universidad Nacional de La Plata, Facultad de Informática, 2021. URL: http://sedici.unlp.edu.ar/handle/10915/129853.

[24] SignAll Technologies. *About SignAll Technologies*. https://www.linkedin.com/company/signall-technologies/about/. 2026.

[25] D. Li, C. Rodriguez, X. Yu y H. Li. «Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison». En: *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*. 2020, págs. 1459-1469.

[26] Hand Talk. *Acessibilidade digital em línguas de sinais*. https://www.handtalk.me/br/. 2025.

[27] HETAH. *Avatar traductor a lengua de señas*. https://hetah.net/avatar. 2024.

[28] R. S. Pressman y B. R. Maxim. *Software Engineering: A Practitioner's Approach*. 8th. McGraw-Hill, 2014.

[29] T. Akiba, S. Sano, T. Yanase, T. Ohta y M. Koyama. «Optuna: A Next-generation Hyperparameter Optimization Framework». En: (2019), págs. 2623-2631.

[30] Qwen Team. «Qwen2.5 Technical Report». En: *arXiv preprint arXiv:2412.15115* (2024).

[31] L. B. Allal, A. Lozhkov, E. Bakouch, L. von Werra y T. Wolf. *SmolLM2: When Small Goes Big*. https://huggingface.co/blog/smollm2. 2024.

[32] Meta. *Llama 3.2 1B Instruct*. https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct. Ficha del modelo. Publicación: 25 de septiembre de 2024. 2024.

[33] K. Papineni, S. Roukos, T. Ward y W.-J. Zhu. «BLEU: a Method for Automatic Evaluation of Machine Translation». En: *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. 2002, págs. 311-318.

[34] C.-Y. Lin. «ROUGE: A Package for Automatic Evaluation of Summaries». En: *Text Summarization Branches Out*. 2004, págs. 74-81.

[35] S. Banerjee y A. Lavie. «METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments». En: *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. 2005, págs. 65-72.

[36] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger e Y. Artzi. «BERTScore: Evaluating Text Generation with BERT». En: *International Conference on Learning Representations*. 2020.

[37] R. Rei, C. Stewart, A. C. Farinha y A. Lavie. «COMET: A Neural Framework for MT Evaluation». En: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. 2020, págs. 2685-2702.

Apéndice A

Vocabulario del clasificador

El clasificador distingue 100 clases. Cada clase cuenta con 100 clips. Miriam Rolls ayudó con la validación del conjunto. El cuadro A.1 separa el alfabeto, los dígitos y el léxico en grupos chicos, y cuenta cuántas clases hay en cada uno. La lista usa la lectura de la etiqueta: no es una traducción al español ni una definición de la seña.

Cuadro A.1: Señas del clasificador por categoría.

Categoría	**Cantidad**	**Señas**
Alfabeto manual	27	A–Z
Dígitos	10	0–9
Interrogativos	6	cómo, cuándo, dónde, qué, quién, cuántos
Afirmación y negación	2	sí, no
Valoración	2	bien, mal
Pronombres	5	yo, vos, él o ella, nosotros, ellos
Posesivos	2	mío, tuyo
Saludos y cortesía	3	hola, chau, gracias
Tiempo	6	pasado, futuro, día, hora, ahora u hoy, ayer
Días de la semana	7	lunes, martes, miércoles, jueves, viernes, sábado, domingo
Lugares	4	lugar, casa, calle, plaza
Identificación	5	nombre, apellido, documento, número, años
Parentesco	6	familia, mamá, papá, hermano o hermana, hijo o hija, esposo o esposa
Partes del cuerpo	3	brazo, cara, ojo
Objeto	1	cuchillo
Verbos	8	tener, poder, vivir, vivir en, ver, llamar, llevar, repetir
Auxilio	3	ayudar, robar, policía
Total	**100**	

Apéndice B

Ramas de entrenamiento y de servicios

El programa que se instala en la llamada está en la rama principal y se describe en la Sección 6.1. Este apéndice reúne las otras ramas. El detalle de directorios, módulos y convenciones de nombres queda en el `README` de cada una.

B.1. Repositorio de entrenamiento del clasificador

Este repositorio entrena y evalúa el clasificador de señas aisladas. Vive en la rama `scratch-mediapipe-v3`. El archivo de pesos que sale de ese entrenamiento es el que después carga el motor local.

Todo el código está en `src/`. `train.py` ajusta el modelo y guarda el peso, `tune_optuna.py` busca hiperparámetros, y `preprocessing.py` convierte los videos en coordenadas. `camera.py` mide el acierto frente a la cámara. La arquitectura está en `model_arch.py`. `config.py` fija las rutas, las 100 clases y la ventana de 16 fotogramas. La extracción usa el Holistic del navegador, el mismo paquete que la extensión, a través de `holistic_web.py`. Los videos quedan en `dataset/` y las coordenadas en `dataset_landmarks_16frames/`. Los pesos y los registros salen a `src/model_new_2026_10_20/`, con el checkpoint `tinyskeleton_best.pth`.

B.2. Servicio semántico en la nube

Este repositorio es el servicio que el motor local consulta para traducir. Atiende las dos direcciones, de glosas hacia español y de español hacia glosas, con los binarios que entregan los repositorios de entrenamiento. Su tarea es responder un enunciado ya armado, y no forma parte de lo que se instala en la computadora de quien llama.

El arranque está en `run_semantic_server.py`. El servicio vive en `src/semantic/`: `api.py` expone las rutas, `service.py` separa los dos canales y `engine.py` ejecuta el binario. Los pesos no comparten carpeta. `models/sign/` guarda el canal de glosas hacia español y `models/hearing/` el canal inverso. En cada una hay un binario principal, un respaldo y el `sys_prompt.txt` de esa dirección.

B.3. Entrenamiento del canal directo

Este repositorio ajusta y evalúa los modelos que pasan de glosas a una oración en español. El binario que resulta de ese entrenamiento es el que después carga el servicio en la nube para el canal directo.

Todo el código está en `src/semantic/`. `train.py` ajusta un modelo y exporta el binario, `eval_trial.py` lo compara sobre oraciones fuera del corte, y `models.py` es el catálogo: sumar una familia es agregar una entrada ahí. El corpus está en `dataset_glosses.json`, las oraciones de prueba en `dataset_trial.json` y las instrucciones del modelo en `prompts/sys_prompt.txt`. Los pesos y los registros salen a `outputs/`. Cada familia se identifica con un id estable, como `qwen2.5-1.5b` o `llama-3.2-1b`, y ese mismo id nombra su carpeta de salida.

B.4. Entrenamiento del canal inverso

Este repositorio ajusta y evalúa los modelos que pasan de una oración en español a una secuencia de glosas. El corpus y el binario son propios de esta dirección, y el servicio en la nube los usa para el canal inverso.

La disposición repite la del canal directo, también bajo `src/semantic/` y también con los módulos en inglés: `train.py` entrena, `eval_trial.py` compara y `config.py` reúne rutas, hiperparámetros y la lista de modelos. El corpus, el archivo de prueba y el prompt ocupan los mismos nombres de archivo que en el otro laboratorio. Los id de modelo también coinciden, de modo que `qwen2.5-1.5b` designa la misma familia en las dos direcciones, con pesos distintos.

B.5. Servicio de poses del avatar

Este repositorio publica los archivos de pose que el motor local pide para dibujar el avatar. No entrena modelos ni traduce texto. El cliente de la videollamada solicita esas poses a este servicio.

El código cabe en `app/`: `main.py` arranca el servicio, `config.py` fija rutas y tipos de archivo, y `fingerspell.py` resuelve el alfabeto manual. Las poses están en `poses/`. El nombre del archivo, sin extensión, es la glosa ya normalizada: mayúsculas, sin tildes y en ASCII. Dentro de una palabra la eñe pasa a ene, y la letra eñe, cuando es la glosa entera, se pide como `ENIE`.

Apéndice C

Arquitectura escalable de producción en la nube

La Figura C.1 pertenece a la Sección 5.6. La hoja va en horizontal para que el ingreso, el traductor y el servicio de archivos queden en una sola vista. El diagrama describe la proyección para varias videollamadas a la vez.

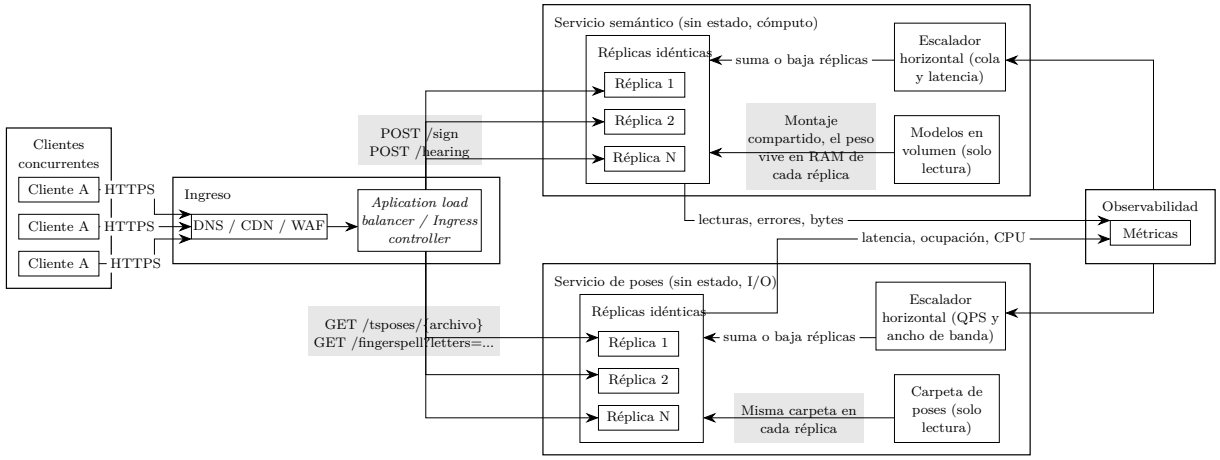


Figura C.1: Diagrama de la arquitectura del servidor, balanceo de carga y escalabilidad de servicios sin estado.